ВСЕРОССИЙСКИЙ КОНКУРС ДОСТИЖЕНИЙ ТАЛАНТЛИВОЙ МОЛОДЁЖИ

«НАЦИОНАЛЬНОЕ ДОСТОЯНИЕ РОССИИ»

Направление:
ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ

Тема:
Экспериментальная оценка эффективности искусственного интеллекта в генерации кода для доменно-специфичных платформ (на примере 1С:Предприятие 8)

Автор: Андреев Данила Сергеевич

Научный руководитель: Гальцкова Юлия Михайловна

Место выполнения работы: ГАПОУ по «Пензенский Колледж Информационных и Промышленных Технологий (ИТ-Колледж)

2026

СОДЕРЖАНИЕ

Введение 5
ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И ПРОБЛЕМАТИКА ИССЛЕДОВАНИЯ 8
1.1. Современные языковые модели: возможности и ограничения 8
1.2. Специфика платформы 1С:Предприятие как доменно-специфичной среды 8
1.3. Проблема оценки качества генерации для DSL 9
1.4. Model Context Protocol как способ интеграции контекста метаданных 10
1.5. Выводы и постановка задачи на разработку методики оценки 12
ГЛАВА 2. РАЗРАБОТКА МЕТОДИКИ И ПРОГРАММНОГО КОМПЛЕКСА ОЦЕНКИ 13
2.1. Концепция метрики SMOP (Syntax, Meaning, Optimization, Platform) 13
2.1.1 Обоснование выбора критериев 13
2.1.2 Дискретная шкала оценивания и её валидация 13
2.1.3 Интегральный показатель качества Q 15
2.2. Обоснование разработки собственного программного комплекса 15
2.2.1. Ограничения существующих решений 15
2.2.2. Требования к инструментарию научного эксперимента 16
2.2.3. Выводы по необходимости разработки собственного комплекса 16
2.3. Архитектура программного комплекса GenLab-1C 17
2.3.1 Общая архитектура комплекса и поток данных 17
2.3.2 Open Source научное ядро (AI-1C-Code-Gen-Benchmark) 18
2.3.2.1 Технологический стек ядра 18
2.3.2.2 Структура и конфигурационные файлы 19
2.3.2.3 Модуль генерации и взаимодействия с LLM 19
2.3.2.4 Модуль экспертной оценки SMOP 20
2.3.2.5 Модуль статистики и визуализации 21
2.3.2.6 Интерфейсы научного ядра 22
2.3.3 Веб-платформа GenLab-1C 23
2.3.3.1 Технологический стек 23
2.3.3.2 Управление настройками эксперимента 24
2.3.3.3 Управление тестовыми заданиями 24
2.3.3.4 Коллаборативная экспертная оценка 26
2.3.3.5. Визуализация результатов и дашборды 26

2.3.3.6. Веб документация методики SMOP ... 27
2.3.4 Интеграция ядра и веб-платформы ... 28
2.4. Тестовая инфраструктура ... 29
2.5. Дизайн эксперимента ... 29
2.5.1 Тестовый набор задач ... 29
2.5.2 Исследуемые модели и параметры генерации ... 30
2.5.3 Сводные параметры эксперимента ... 32
ГЛАВА 3. Результаты экспериментальной апробации ... 33
3.1. Категория А (задачи без контекста метаданных) ... 33
3.1.1 Общие показатели ... 33
3.1.2 Детальный анализ метрик ... 34
3.1.3 Ранжирование моделей по качеству ... 35
3.2. Категория Б (платформенно-ориентированные задачи) ... 36
3.2.1 Общие показатели ... 36
3.2.2 Детальный анализ метрик ... 37
3.2.3 Ранжирование моделей по качеству ... 38
3.3. Сравнительный анализ категорий А и В ... 39
3.3.1 Влияние МСР на качество генерации ... 39
3.3.2 Анализ причин падения качества ... 40
3.3.3 Выводы по сравнительному анализу ... 40
3.4. Ограничения исследования ... 41
ЗАКЛЮЧЕНИЕ ... 43
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ ... 45

АННОТАЦИЯ

В настоящей работе представлены результаты экспериментального исследования эффективности современных ИИ-инструментов в генерации кода на встроенном языке платформы 1С:Предприятие 8 (BSL — Business Script Language). Проведен сравнительный анализ трех универсальных языковых моделей (Claude Opus 4.5, GPT-5.2 Codex, Gemini 3 Flash) с применением технологии Model Context Protocol (MCP).

Основным результатом работы является разработка методики многокритериальной оценки SMOP (Syntax, Meaning, Optimization, Platform) и создание программного комплекса GenLab-1C, включающего научное ядро для проведения экспериментов и веб-платформу для коллективной экспертной работы. Методика SMOP оценивает качество сгенерированного кода по четырём критериям: синтаксическая корректность, семантическая корректность, оптимальность и глубина интеграции с платформенными метаданными. Эксперимент выполнен на конфигурации «Управление торговлей 11» с использованием набора тестовых заданий, охватывающих как стандартные алгоритмические задачи, так и задачи, требующие понимания внутреннего устройства конфигурации.

Научная новизна работы заключается в разработке специализированной метрики качества для DSL-сред и создании открытого воспроизводимого инструментария для проведения экспериментов. Данная работа позволяет не только оценить влияние интеграции контекста метаданных через MCP на качество генерации кода, но и предоставляет исследовательскому сообществу методологию и программные средства для проведения аналогичных исследований.

Практическая значимость исследования заключается в предоставлении рекомендаций для разработчиков 1С, интеграторов ERP-систем и исследователей, изучающих применение больших языковых моделей в доменно-специфических средах. Разработанная методология и открытое программное ядро могут быть адаптированы для оценки других DSL и платформ, а веб-платформа GenLab обеспечивает возможность коллективной экспертной работы.

ВВЕДЕНИЕ

Актуальность: в последние годы большие языковые модели (Large Language Models — LLM) достигли значительных успехов в задачах генерации программного кода. Согласно данным бенчмарка SWE-bench Verified, современные модели способны решать до 76–81% реальных задач разработки для универсальных языков программирования, таких как Python и Java [1, 2]. Данный прогресс обусловлен как увеличением размера обучающих выборок, так и совершенствованием архитектур нейронных сетей.

Однако применение LLM для генерации кода на доменно-специфических языках (Domain-Specific Languages — DSL) по-прежнему сталкивается с рядом ограничений. Одним из характерных представителей таких языков является встроенный язык платформы 1С:Предприятие (Business Scripting Language — BSL), который, согласно данным аналитического агентства TAdviser, решения «1С» покрывают около 50% российского рынка ERP-систем (достигая 90% в сегменте малого бизнеса), при этом в 2023–2024 годах наблюдается устойчивый рост спроса со стороны средне-крупных компаний и госсектора, а сегмент поддержки может увеличиться в 1,5–2 раза [3]. Специфика BSL заключается в его тесной связи с метамоделью конкретной конфигурации: эффективный код невозможен без понимания структуры документов, регистров, справочников и их взаимосвязей. Кроме того, код на BSL крайне слабо представлен в общедоступных обучающих датасетах — по оценкам автора, его доля в корпусе The Stack, содержащем 3 ТБ исходного кода на 358 языках, не превышает сотых долей процента [4].

Данная ситуация порождает фундаментальную проблему: существующие методы оценки качества генерации кода, разработанные для универсальных языков (HumanEval, MBPP, SWE-bench), не учитывают специфику DSL-сред и не позволяют получить объективную картину применимости LLM в таких предметных областях, как 1С:Предприятие. Отсутствие специализированной методики и программного инструментария затрудняет как исследование возможностей современных моделей, так и их практическое внедрение в процессы разработки конфигураций.

Технология Model Context Protocol (MCP), представленная в ноябре 2024 года, открывает новые перспективы для интеграции LLM с метаданными конкретных информационных систем [5]. Данный протокол обеспечивает динамическое предоставление модели структурированного контекста, что потенциально способно компенсировать недостаток обучающих данных по BSL. Однако для количественной оценки эффективности как самой технологии MCP, так и различных языковых моделей в контексте платформы 1С:Предприятие, требуется разработка специализированного инструментария, обеспечивающего воспроизводимость экспериментов и объективность измерений.

Степень разработанности проблемы. Академические исследования в области генерации кода для языка BSL практически отсутствуют ввиду узкой специализации платформы. Существующие публикации профессионального сообщества [6, 7] носят преимущественно описательный характер и фиксируют отдельные проблемы, такие как "галлюцинации" моделей, использование несуществующих методов платформы, некорректные имена реквизитов и смешение синтаксиса SQL с языком запросов 1С. Систематические же исследования с применением формализованных метрик и статистической обработки данных до настоящего времени не проводились.

Проблема исследования заключается в отсутствии валидированной методики и программного инструментария для объективной количественной оценки качества кода, генерируемого большими языковыми моделями для платформы 1С:Предприятие, что препятствует как сравнительному анализу существующих LLM-решений, так и оценке эффективности технологий контекстной интеграции, в частности MCP.

Цель: разработка методики многокритериальной оценки качества генерации кода большими языковыми моделями для платформы 1С:Предприятие, создание открытого программного

комплекса, реализующего данную методику, и его апробация в ходе экспериментального исследования эффективности современных LLM с применением технологии MCP.

Задачи:

1. провести анализ современных подходов к генерации кода большими языковыми моделями и выявить ограничения их применимости в доменно-специфических средах на примере платформы 1С:Предприятие;
2. разработать многокритериальную метрику SMOP (Syntax, Meaning, Optimization, Platform), адаптированную к специфике платформы и позволяющую дифференцированно оценивать различные аспекты качества сгенерированного кода.
3. создать программный комплекс GenLab-1C, включающий открытое научное ядро для проведения воспроизводимых экспериментов и веб-платформу для коллективной экспертной работы.
4. спроектировать эксперимент, включающий тестовый набор задач двух категорий (алгоритмические и платформенно-зависимые), и выполнить сравнительное исследование трёх современных языковых моделей (Claude Opus 4.5, GPT-5.2 Codex, Gemini 3 Flash) в сценариях с применением технологии MCP и без него.
5. провести статистический анализ полученных данных, оценить влияние MCP на качество генерации и сформулировать рекомендации по применению LLM в задачах разработки для платформы 1С:Предприятие.

Предмет: методы и метрики оценки качества кода, генерируемого большими языковыми моделями для доменно-специфичной платформы 1С:Предприятие, а также влияние технологии контекстной интеграции MCP на результаты генерации.

Объект: процесс генерации программного кода большими языковыми моделями.

Научная новизна:

1. впервые предложена и обоснована многокритериальная метрика SMOP для оценки качества генерации кода в среде 1С:Предприятие, учитывающая не только синтаксическую и семантическую корректность, но и оптимизационную составляющую, а также глубину интеграции с платформенными метаданными;
2. разработан открытый программный комплекс GenLab-1C, обеспечивающий полный цикл экспериментального исследования: от генерации кода через OpenRouter API до экспертной оценки с применением TUI-интерфейса и коллективной работы через веб-платформу с автоматической статистической обработкой результатов;
3. впервые получены количественные данные о сравнительной эффективности ведущих LLM (Claude Opus 4.5, GPT-5.2 Codex, Gemini 3 Flash) при генерации кода BSL, а также статистически значимые оценки влияния технологии MCP на качество результатов в зависимости от типа решаемых задач.

Период исследования: декабрь 2025 – февраль 2026.

Практическая значимость: определяется созданием открытого инструментария, который может быть использован для:

— объективного сравнения различных LLM при выборе ИИ-ассистента для задач 1С-разработки;

— выявления типовых ошибок и "галлюцинаций" моделей с целью их дальнейшего устранения;

— оценки эффективности технологий контекстной интеграции, подобных MCP;

— проведения дальнейших исследований в области применения ИИ в доменно-специфических средах как научным сообществом, так и практикующими разработчиками.

Объем и структура работы: аннотация, введение, 3 главы, заключение, 14 информационных источников.

Методы исследования: в работе использованы методы системного анализа, теоретического обобщения, экспериментального проектирования, экспертного оценивания, статистической обработки данных (расчёт описательных статистик, доверительных интервалов, корреляционный анализ, оценка межэкспертной надёжности с применением формулы Каппы Коэна и т.д.), а также методы объектно-ориентированного программирования и веб-разработки.

Проблема: низкая эффективность языковых моделей для генерации кода на 1С из-за ограничений данных и специфики среды, отсутствие методики и инструментария для оценки.

Гипотеза: разработанная методика SMOP позволяет проводить дифференцированную оценку качества генерации кода и выявлять узкие места современных LLM при работе с BSL. Применение технологии MCP значимо повышает интегральный показатель качества Q, причём наибольший прирост наблюдается для задач, требующих глубокого понимания структуры метаданных конфигурации.

ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И ПРОБЛЕМАТИКА ИССЛЕДОВАНИЯ

1.1. Современные языковые модели: возможности и ограничения

Большие языковые модели (Large Language Models — LLM) достигли значительных результатов в задачах генерации программного кода. По данным бенчмарка SWE-bench Verified, модели Claude Opus 4.5, GPT-5.2 Codex и Gemini 3 Flash успешно решают 76–81% реальных задач из GitHub-issues для универсальных языков программирования [1, 2]. Эти модели обучены на массивах данных, включающих миллиарды строк кода на Python, Java, JavaScript и других широко распространённых языках.

Однако возможности LLM при работе с доменно-специфическими языками (Domain-Specific Languages, DSL) существенно ограничены. Основная причина — малая представленность DSL в открытых наборах данных. Датасет The Stack содержит 3 ТБ исходного кода на 358 языках программирования, однако объем кода для доменно-специфического языка BSL, используемого в 1С:Предприятие, в открытых источниках либо не выделен, либо минимален [3].

Помимо количественного дефицита обучающих данных, существует качественная проблема. DSL, включая BSL, тесно связаны с платформенными метаданными: эффективный код невозможен без понимания структуры объектов конфигурации — документов, регистров, справочников. Модель, не имеющая доступа к этой информации, вынуждена «угадывать» имена объектов и их свойства, что приводит к генерации синтаксически корректного, но семантически ошибочного кода — так называемым «галлюцинациям».

Академические исследования генерации кода для языка BSL практически отсутствуют ввиду узкой специализации платформы. В связи с этим в качестве источников эмпирических данных использованы публикации профессионального сообщества разработчиков 1С. Применительно к платформе 1С:Предприятие проблемы генерации проявляются в характерных ошибках [4, 5]:

— использование несуществующих методов платформы;
— некорректные имена реквизитов и регистров;
— смешение синтаксиса SQL и языка запросов 1С;
— непонимание событийной модели платформы;
— игнорирование особенностей работы с транзакциями;
— некорректная работа с типами данных, специфичными для 1С.

Таким образом, несмотря на высокую эффективность современных LLM в генерации кода на массовых языках программирования, их применимость к доменно-специфическим языкам, таким как BSL, существенно ограничена как дефицитом обучающих данных, так и необходимостью понимания платформенного контекста.

1.2. Специфика платформы 1С:Предприятие как доменно-специфичной среды

Платформа 1С:Предприятие занимает уникальное положение на российском рынке программного обеспечения. Согласно данным аналитического агентства TAdviser, решения на базе 1С покрывают около 50% отечественного рынка ERP-систем, причём в сегменте малого бизнеса этот показатель достигает 90% [3]. Широкое распространение платформы обуславливает высокий спрос на разработку и сопровождение конфигураций, что делает актуальным исследование возможностей автоматизации этих процессов с применением искусственного интеллекта.

Ключевая особенность платформы 1С:Предприятие заключается в её двухслойной архитектуре. Платформа предоставляет исполняющую среду и набор базовых механизмов (учёт, отчёты, интерфейсы), в то время как прикладная логика реализуется в конфигурациях — метаданных, описывающих структуры данных, формы, алгоритмы и бизнес-процессы. Встроенный язык программирования BSL (Business Scripting Language) предназначен именно для написания прикладной логики в контексте конкретной конфигурации.

Специфика BSL как доменно-специфического языка проявляется в нескольких аспектах:

1. Тесная связь с метаданными. Код на BSL постоянно обращается к объектам конфигурации: справочникам, документам, регистрам, перечислениям. Например, строка Справочники.Контрагенты.НайтиПоКоду("0001") предполагает наличие в конфигурации справочника "Контрагенты" с соответствующим реквизитным составом.

2. Декларативно-процедурный характер. Помимо процедурного кода, значительная часть логики реализуется через декларативные механизмы — запросы на языке запросов 1С, который имеет собственную специфику, отличную от SQL.

3. Событийно-ориентированная модель. Разработка ведётся в парадигме обработки событий (при проведении документа, при записи справочника, при открытии формы), что требует понимания жизненного цикла объектов.

4. Ограниченная изоляция от платформы. Многие механизмы (блокировки, транзакции, работа с регистрами) реализуются неявно, и их некорректное использование может приводить к трудно диагностируемым ошибкам.

Перечисленные особенности делают генерацию кода для 1С принципиально более сложной задачей по сравнению с универсальными языками. Модель должна не просто владеть синтаксисом BSL, но и понимать структуру конкретной конфигурации, а также семантику платформенных механизмов.

1.3. Проблема оценки качества генерации для DSL

Для оценки качества генерации кода на универсальных языках программирования разработано множество бенчмарков и метрик. Наибольшее распространение получили:
— HumanEval — набор из 164 задач на Python с тестами для проверки корректности;
— MBPP (Mostly Basic Python Problems) — 974 задачи начального уровня;
— SWE-bench — реалистичные задачи, основанные на реальных pull request из GitHub [1, 2].
Данные бенчмарки используют автоматическую проверку через запуск тестов, что обеспечивает объективность и воспроизводимость оценки.

В 2025 году появились первые специализированные бенчмарки для оценки генерации кода на платформе 1С:Предприятие. 1С Code Bench [11] представляет собой набор из 20 практических задач на языке BSL, выполняемых в демонстрационной конфигурации «Управляемое приложение». Каждая задача оформлена в виде внешней обработки (.epf), содержащей текст задания, тестовое окружение и функцию автоматической проверки `ЗадачаРешена()`, возвращающую булево значение. Основной метрикой является pass@k — доля задач, для которых хотя бы одно из k сгенерированных решений прошло все тесты. BenchLabs [12] реализует аналогичный подход с использованием фреймворка модульного тестирования YAXUNIT и предоставляет открытый рейтинг моделей (Claude, GPT, GigaChat, Qwen, DeepSeek и др.). Исследовательский проект Харина В. [13] содержит набор промптов, эталонных решений и скелетную конфигурацию для ручного тестирования открытых моделей в задачах разработки и архитектурного проектирования 1С.

Указанные бенчмарки представляют значительный вклад в формирование инструментария оценки LLM для платформы 1С, однако в контексте настоящего исследования они обладают рядом существенных ограничений.

Во-первых, бинарная метрика pass@k не обеспечивает дифференциации источников ошибок. Решение, содержащее единственную опечатку в имени реквизита, и решение, построенное на полностью вымышленной структуре конфигурации, получают одинаковую оценку «не пройдено». Между тем для целей сравнительного анализа подходов к интеграции контекста (в частности, MCP [5] и без него) принципиально важно различать, на каком именно уровне — синтаксическом, семантическом, платформенном или оптимизационном — генерация оказалась недостаточной.

Во-вторых, существующие бенчмарки не выделяют платформенную интеграцию в качестве самостоятельного критерия оценки. Корректность обращения к объектам метаданных конкретной конфигурации — регистрам, реквизитам, табличным частям — оценивается косвенно, через факт прохождения или непрохождения теста. Это делает невозможным количественное измерение эффекта от предоставления модели контекста метаданных через MCP-сервер [7, 10], поскольку именно метрика платформенной интеграции является целевой для данного исследования.

В-третьих, метрика pass@k не учитывает оптимальность решения. Код, содержащий запросы к базе данных в цикле, избыточные обращения к серверу или иные антипаттерны [6], может успешно пройти функциональный тест, но быть непригодным к промышленной эксплуатации. Для задач 1С-разработки, где производительность запросов критически важна в условиях многопользовательской работы, оценка оптимальности является необходимым компонентом комплексной методики.

В-четвёртых, тестирование привязано к фиксированной демонстрационной конфигурации, что ограничивает обобщаемость результатов. Конфигурации 1С:Предприятие существенно различаются структурой метаданных [3], и способность модели корректно работать с демонстрационной базой не гарантирует аналогичного результата на промышленных конфигурациях.

Таким образом, существующие бенчмарки решают задачу ранжирования моделей по критерию функциональной корректности, но не обеспечивают многокритериального анализа качества генерации с выделением вклада отдельных аспектов. Наблюдается методологический разрыв: отсутствует валидированная методика, позволяющая (а) дифференцированно оценивать качество генерации кода для платформы 1С по нескольким независимым критериям, (б) количественно измерять эффект от предоставления контекста метаданных и (в) обеспечивать воспроизводимость оценок при проведении сравнительных экспериментов.

1.4. Model Context Protocol как способ интеграции контекста метаданных

Технология Model Context Protocol (MCP) представляет собой открытый стандарт, представленный в ноябре 2024 года для унификации взаимодействия между большими языковыми моделями и внешними системами [5]. Протокол решает проблему «N×M интеграций»: вместо создания отдельных коннекторов для каждой пары «модель — инструмент» MCP предлагает единый интерфейс, через который любая совместимая модель может взаимодействовать с любым совместимым сервером [8].

Архитектура протокола MCP построена на клиент-серверной модели с тремя ключевыми компонентами [8, 9]:

— host (хост) — приложение, использующее LLM (например, Claude Desktop, IDE с ИИ-ассистентом). Хост инициирует соединения, управляет разрешениями и координирует взаимодействие;

— client (клиент) — модуль внутри хоста, выступающий посредником между моделью и серверами;

— server (сервер) — внешний сервис, предоставляющий доступ к данным и инструментам.

MCP определяет три категории примитивов для взаимодействия:

— resources (ресурсы) — структурированные данные, которые сервер предоставляет модели (файлы, записи баз данных, метаданные);

— tools (инструменты) — функции, которые модель может вызывать для выполнения действий;

— prompts (подсказки) — шаблоны взаимодействия, определяющие типовые сценарии использования сервера.

Для доменно-специфических сред, таких как 1С:Предприятие, MCP открывает принципиально новую возможность — предоставление модели структурированного контекста метаданных конфигурации без необходимости включения этой информации в обучающую

выборку. Сервер может динамически передавать описания объектов (документов, регистров, справочников), их реквизитов и связей, что позволяет модели генерировать семантически корректный код для конкретной информационной базы.

Практическая реализация взаимодействия языковых моделей с платформой 1С:Предприятие через протокол MCP осуществляется с помощью открытого проекта 1c_mcp [10]. Проект распространяется под лицензией MIT и предоставляет готовую инфраструктуру для подключения MCP-совместимых клиентов (Claude Desktop, Cursor, VS Code) к информационным базам 1С.

Проект состоит из двух основных компонентов:

— расширение 1С (.cfe) — серверная часть, реализующая логику обработки MCP-запросов на стороне платформы. Расширение публикуется как HTTP-сервис mcp_APIBackend и обрабатывает JSON-RPC запросы от клиентов. Готовая сборка расширения находится в каталоге build/MCP_Сервер.cfe;

— python-прокси — промежуточный компонент, транслирующий MCP-протокол в JSON-RPC вызовы к HTTP-сервису 1С. Прокси решает две инфраструктурные задачи: поддержку транспорта STDIO для локальных клиентов и реализацию OAuth2-аутентификации с передачей учётных данных в 1С через Basic Auth.

Решение реализует все три категории примитивов протокола MCP:

— tools (инструменты) — функции для получения метаданных конфигурации, выполнения запросов, работы с объектами. Разработчик может расширять набор инструментов, создавая обработки в подсистеме mcp_КонтейнерыИнструментов и реализуя методы ДобавитьИнструменты() и ВыполнитьИнструмент();

— resources (ресурсы) — статические данные конфигурации: структура метаданных, описания объектов, шаблоны кода.

— prompts (подсказки) — предопределённые сценарии взаимодействия с конфигурацией.

Проект поддерживает несколько вариантов подключения:

— прямое подключение — MCP-клиент обращается непосредственно к HTTP-сервису 1С. Ограничение: требуется публикация без аутентификации на уровне платформы;

— через Python-прокси (рекомендуемый) — прокси обеспечивает поддержку STDIO-транспорта и OAuth2-аутентификации. Требования: Python версии 3.11 и выше, платформа 1С:Предприятие версии 8.3.20 и выше;

— Docker-контейнер — изолированный запуск прокси-сервера для упрощения развёртывания.

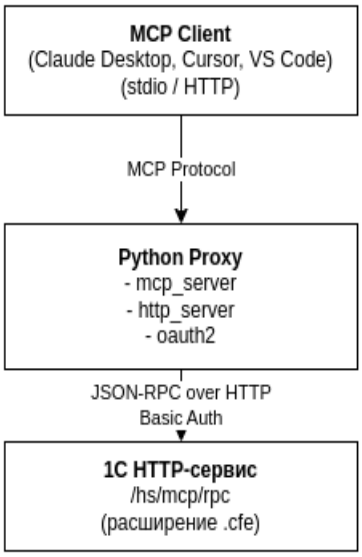


Рисунок 1.1 — схема взаимодействия компонентов между MCP Client и 1С:Предприятие

Проект vladimir-kharin/1c_mcp предоставляет техническую основу для экспериментальной оценки влияния MCP-контекста на качество генерации кода. Благодаря расширяемой архитектуре возможно создание специализированных инструментов, возвращающих метаданные конфигурации УТ11: структуру регистров накопления, реквизиты документов, описания табличных частей. Это позволяет языковой модели получать актуальную информацию о конфигурации непосредственно в процессе генерации кода.

1.5. Выводы и постановка задачи на разработку методики оценки

Проведённый теоретический анализ позволяет сформулировать следующие ключевые выводы:

1. Современные языковые модели демонстрируют высокую эффективность при генерации кода на универсальных языках программирования, однако их применимость к доменно-специфическому языку BSL ограничена как дефицитом обучающих данных [4], так и необходимостью понимания контекста конкретной конфигурации.

2. Специфика платформы 1С:Предприятие (тесная связь с метаданными, событийно-ориентированная модель, наличие декларативных механизмов) требует особого подхода к оценке качества генерируемого кода, учитывающего не только синтаксическую и семантическую корректность, но и оптимальность, а также глубину интеграции с платформенными механизмами.

3. Существующие бенчмарки для универсальных языков (HumanEval, MBPP, SWE-bench [1, 2]) неприменимы к BSL в силу их нечувствительности к платформенному контексту. Специализированные бенчмарки для 1С (1С Code Bench [11], BenchLabs [12]) представляют важный шаг в направлении оценки LLM для данной платформы, однако используют бинарную метрику pass@k, не позволяющую дифференцировать источники ошибок и измерять вклад отдельных аспектов качества — синтаксической корректности, семантического соответствия, платформенной интеграции и оптимальности.

4. Технология MCP [5, 8, 9] предоставляет перспективный механизм интеграции контекста метаданных в процесс генерации, однако количественная оценка её эффективности для задач 1С-разработки невозможна с использованием существующих бенчмарков, поскольку они не выделяют платформенную интеграцию как самостоятельный измеримый критерий.

Таким образом, возникает необходимость в разработке специализированной методики оценки качества генерации кода для платформы 1С:Предприятие, которая должна:

— учитывать многокритериальную природу качества (синтаксис, семантика, оптимальность, платформенная интеграция);

— дополнять существующие бенчмарки [11, 12], обеспечивая градуированную оценку вместо бинарной;

— оперировать формализованной шкалой, обеспечивающей воспроизводимость оценок;

— допускать статистическую обработку результатов для получения количественных выводов;

— быть применимой как для оценки отдельных моделей, так и для сравнения различных подходов к интеграции контекста (в частности, с применением MCP [5] и без него).

Разработка такой методики и создание программного комплекса, реализующего её, составляет содержание следующей главы настоящей работы.

ГЛАВА 2. РАЗРАБОТКА МЕТОДИКИ И ПРОГРАММНОГО КОМПЛЕКСА ОЦЕНКИ

2.1. Концепция метрики SMOP (Syntax, Meaning, Optimization, Platform)

2.1.1 Обоснование выбора критериев

Для оценки качества сгенерированного кода разработана система из четырёх метрик, образующих аббревиатуру SMOP: Syntax (синтаксическая корректность), Meaning (семантическая корректность), Optimization (оптимальность), Platform (платформенная интеграция). Выбор именно этих четырёх критериев обусловлен спецификой платформы 1С:Предприятие как доменно-специфичной среды.

В таблице 2.1 представлено определение каждой метрики.

Таблица 2.1. — Система метрик оценки качества генерации кода

Метрика	Обозначение	Определение
Синтаксическая корректность	S	Способность кода компилироваться в среде 1С:Предприятие
Семантическая корректность	M	Соответствие логики кода требованиям задания
Платформенная интеграция	P	Корректность использования объектов метаданных платформы
Оптимальность	O	Соответствие кода стандартам и лучшим практикам разработки 1С

Синтаксическая корректность (S) является базовым критерием — код, содержащий синтаксические ошибки, не может быть исполнен платформой. Семантическая корректность (M) отражает понимание моделью поставленной задачи. Платформенная интеграция (P) — ключевая метрика для данного исследования — демонстрирует способность модели работать с конкретной конфигурацией, а не абстрактным языком. Именно по этой метрике ожидается наибольший прирост при использовании MCP, поскольку протокол предоставляет модели актуальную информацию о структуре метаданных. Оптимальность (O) дополняет картину, оценивая не только работоспособность, но и качество решения с точки зрения опытного разработчика.

2.1.2 Дискретная шкала оценивания и её валидация

Каждая метрика оценивается по дискретной шкале от 0 до 10 баллов с шагом 2 балла: [0, 2, 4, 6, 8, 10].

Чётная шкала исключает возможность присвоения «среднего» значения и требует от эксперта однозначного решения о качестве кода. Шаг в 2 балла выбран намеренно: он вынуждает эксперта принимать однозначное решение, избегая соблазна поставить «среднюю» оценку в неопределённых случаях.

Для каждого балла определены конкретные критерии, что снижает субъективность экспертной оценки и обеспечивает воспроизводимость результатов. В таблицах 2.2–2.5 представлены описательные критерии для каждой метрики.

Таблица 2.2. — Критерии оценки синтаксической корректности (S)

Балл	Критерий
10	Код компилируется без ошибок и предупреждений

8	Код компилируется, присутствуют 1–2 незначительных предупреждения
6	Код компилируется после исправления 1–2 очевидных опечаток
4	Требуется исправление 3–5 синтаксических ошибок
2	Требуется исправление более 5 ошибок, однако общая структура кода понятна
0	Код не компилируется, требуется существенная переработка

Таблица 2.3 — Критерии оценки семантической корректности (M)

Балл	Критерий
10	Код полностью выполняет поставленную задачу, все требования реализованы
8	Основная логика реализована верно, не выполнено одно второстепенное требование
6	Основная логика верна, не реализованы 2–3 требования или не обработаны краевые случаи
4	Реализована только часть требуемой логики (более 50%)
2	Реализовано менее 50% требуемой логики
0	Код не выполняет поставленную задачу

Таблица 2.4 — Критерии оценки платформенной интеграции (P)

Балл	Критерий
10	Все объекты метаданных и методы платформы использованы корректно
8	Присутствуют 1–2 неточности в именах реквизитов или методов, легко исправимые
6	Допущены ошибки в работе с типами данных или структурой объектов конфигурации
4	Использованы несуществующие реквизиты, регистры или методы (до 30% обращений)
2	Более 30% обращений к несуществующим объектам метаданных или методам платформы
0	Код построен на полностью вымышленной структуре конфигурации

Таблица 2.5 — Критерии оценки оптимальности (O)

Балл	Критерий
10	Код соответствует стандартам разработки 1С, оптимален по производительности
8	Незначительные отклонения от стандартов, не влияющие на производительность
6	Код работоспособен, но имеются явные возможности оптимизации
4	Присутствуют неэффективные решения: запросы в цикле, избыточные обращения к базе данных
2	Множественные антипаттерны, код будет работать неприемлемо медленно
0	Критические проблемы производительности или архитектуры решения

Валидация шкалы проводилась в ходе пилотного оценивания с участием трёх экспертов — разработчиков 1С с опытом от 2 лет. По результатам пилотной сессии в описания были внесены уточнения, направленные на повышение однозначности интерпретации.

2.1.3 Интегральный показатель качества Q

Итоговая оценка качества сгенерированного кода рассчитывается как среднее арифметическое четырёх метрик:

$$Q = \frac{S + M + O + P}{4}$$

Выбор равных весов обусловлен отсутствием эмпирических данных об относительной значимости отдельных метрик для задач 1С-разработки и является консервативным допущением, принятым в рамках данного исследования.

Интегральный показатель Q принимает значения от 0 до 10 и интерпретируется следующим образом:

$Q \geq 8$ — высокое качество, код пригоден к использованию с минимальными доработками;

$5 \leq Q < 8$ — приемлемое качество, требуется доработка;

$Q < 5$ — низкое качество, код требует существенной переработки.

Предложенная система метрик позволяет оценить качество кода с разных сторон и выявить конкретные проблемные зоны у различных моделей.

2.2. Обоснование разработки собственного программного комплекса

При планировании эксперимента рассматривалась возможность использования существующих инструментов ИИ-ассистентов, интегрированных в среды разработки: GitHub Copilot, Cursor IDE, ассистенты на базе LLM в среде 1С:Enterprise Development Tools (EDT). Однако анализ показал, что ни один из существующих инструментов не удовлетворяет требованиям научного эксперимента в полной мере.

2.2.1. Ограничения существующих решений

GitHub Copilot — наиболее распространённый ИИ-ассистент для генерации кода. Однако его применение в рамках данного исследования ограничено следующими факторами:

1. «Чёрный ящик». Copilot не предоставляет доступа к параметрам генерации (температура, seed), не позволяет управлять системным промптом и не даёт информации о том, какая именно модель использовалась для генерации конкретного ответа. Для научного эксперимента, требующего воспроизводимости и контролируемости условий, это неприемлемо.

2. Отсутствие API для пакетной обработки. Copilot интегрирован в редакторы кода и не предназначен для программного запуска экспериментов с сотнями генераций.

3. Неизвестный состав обучающих данных. Copilot обучался на коде из публичных репозиториев, но точный состав датасета неизвестен, что не позволяет оценить, не встречались ли тестовые задачи в обучающей выборке.

Cursor IDE предлагает более гибкие возможности, включая выбор модели и настройку параметров, однако:

1. Ориентация на интерактивную работу. Инструмент предназначен для работы разработчика в реальном времени, а не для пакетного запуска экспериментов.

2. Отсутствие встроенных механизмов для экспертной оценки. Cursor не предоставляет инструментов для систематической оценки качества сгенерированного кода по формализованным метрикам.

3. Ограниченная поддержка 1С. На момент проведения исследования полноценная поддержка языка BSL в Cursor отсутствовала.

ИИ-ассистенты в EDT (такие как 1С:Напарник) находятся на ранней стадии развития и, как правило, ограничены подсказками при написании кода, не предоставляя возможностей для генерации законченных решений по текстовому описанию.

2.2.2. Требования к инструментарию научного эксперимента

Проведённый анализ позволил сформулировать требования к программному комплексу для проведения данного исследования:

1. Воспроизводимость. Инструмент должен обеспечивать детерминированность генерации (через управление температурой и seed) и фиксировать все параметры эксперимента для возможности его повторения.
2. Прозрачность. Должен быть открыт исходный код вычислительного ядра, чтобы другие исследователи могли проверить корректность реализации.
3. Масштабируемость. Возможность пакетного запуска десятков и сотен генераций без ручного вмешательства.
4. Поддержка множества моделей. Унифицированный доступ к различным LLM (Claude, GPT, Gemini) через единый интерфейс.
5. Интеграция с MCP. Возможность подключения к MCP-серверу для предоставления модели контекста метаданных.
6. Формализованная экспертная оценка. Встроенные механизмы для оценки кода по методике SMOP, включая поддержку множества экспертов и расчёт межэкспертной надёжности.
7. Статистическая обработка. Автоматический расчёт описательных статистик, доверительных интервалов и визуализация результатов.
8. Открытость для сообщества. Возможность использования другими исследователями для проведения аналогичных экспериментов.

2.2.3. Выводы по необходимости разработки собственного комплекса

Ни один из существующих инструментов не удовлетворяет сформулированным требованиям в полной мере. GitHub Copilot и Cursor IDE представляют собой «чёрные ящики», ориентированные на решение прикладных задач разработки, а не на проведение научных экспериментов. Они не обеспечивают необходимого уровня контроля, воспроизводимости и прозрачности.

В связи с этим было принято решение о разработке собственного открытого программного комплекса GenLab-1C, удовлетворяющего всем требованиям научного эксперимента. Ключевыми принципами разработки стали:

— открытость ядра — исходный код публикуется под лицензией MIT, что обеспечивает возможность проверки и модификации;

— модульность — разделение вычислительной логики и пользовательских интерфейсов;

— воспроизводимость — детерминированная генерация, хеширование ответов, сохранение полной конфигурации эксперимента;

— расширяемость — возможность добавления новых моделей, задач и метрик.

Разработанный комплекс не заменяет существующие инструменты, а дополняет их, предоставляя исследовательскому сообществу специализированное средство для проведения экспериментов в области генерации кода для доменно-специфичных платформ.

2.3. Архитектура программного комплекса GenLab-1C

Программный комплекс GenLab-1C спроектирован как двухкомпонентная система, обеспечивающая разделение ответственности между вычислительным ядром и пользовательскими интерфейсами. Ключевым архитектурным решением является полное разделение вычислительной логики и способов взаимодействия с ней, что гарантирует идентичность результатов независимо от используемого интерфейса.

2.3.1 Общая архитектура комплекса и поток данных

На рисунке 2.1 представлена общая архитектура комплекса.

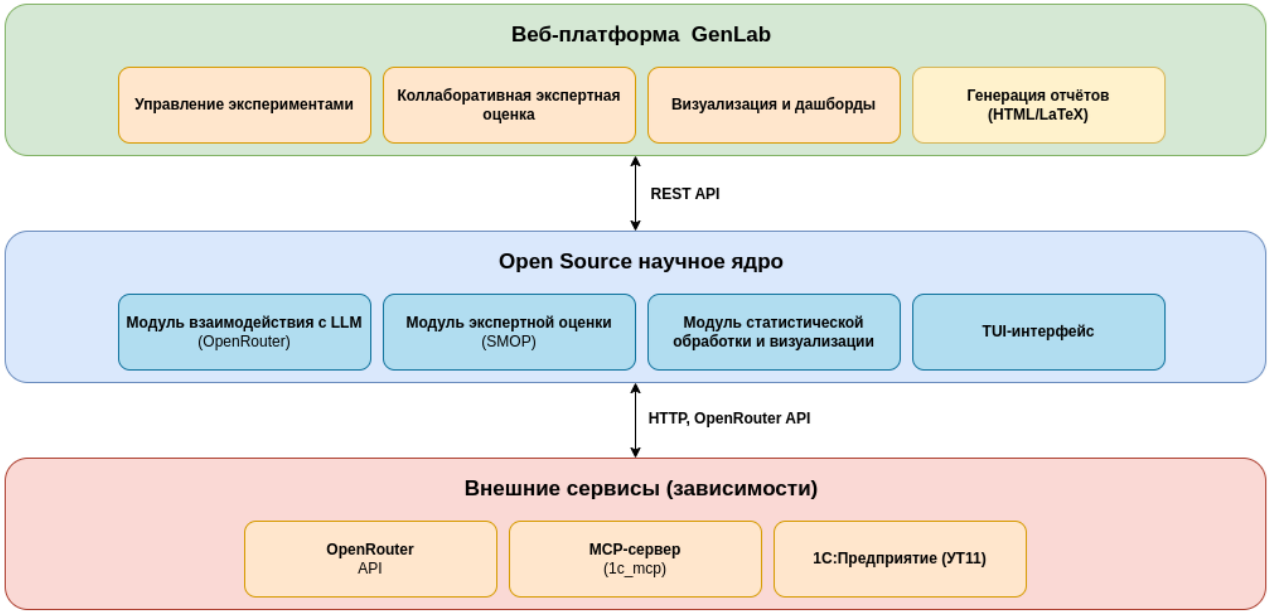


Рисунок 2.1 — общая архитектура комплекса

Комплекс состоит из трёх основных уровней:

1. Внешние сервисы — OpenRouter API (доступ к LLM) и MCP-сервер (доступ к метаданным 1С).
2. Научное ядро — Python-пакет, реализующий всю вычислительную логику.
3. Пользовательские интерфейсы — CLI/TUI для индивидуальной работы и веб-платформа для коллективной.

Поток данных в комплексе организован следующим образом:

1. Конфигурация эксперимента задаётся либо через веб-интерфейс, либо через CLI/TUI ядра. Конфигурация включает: список моделей, набор задач, параметры генерации (температура, seed), сценарий использования MCP.
2. Ядро выполняет генерацию кода, обращаясь к OpenRouter API, сохраняет полученные результаты в структурированном виде и вычисляет хеши ответов для обеспечения воспроизводимости.
3. Сгенерированный код сохраняется в файловой системе (каталог code_outputs/) и регистрируется в JSON-файле результатов эксперимента.
4. Экспертная оценка может проводиться либо через TUI-интерфейс ядра (для индивидуальной работы), либо через веб-платформу (для коллективной оценки несколькими экспертами).
5. Результаты оценки сохраняются в каталоге evaluations/ и служат входными данными для модуля статистической обработки.

6. Модуль статистики рассчитывает описательные статистики, доверительные интервалы, корреляции и показатели межэкспертной надёжности.

7. Модуль визуализации генерирует графики (радарные диаграммы, boxplot, тепловые карты), которые затем интегрируются в отчёты.

8. Генерация отчётов создаёт итоговые артефакты в форматах HTML (интерактивный отчёт), JSON (структурированные данные) и LaTeX (таблицы для публикации).

Описанный поток данных является единым для всех сценариев использования комплекса, что обеспечивает согласованность результатов независимо от выбранного интерфейса.

2.3.2 Open Source научное ядро (AI-1C-Code-Gen-Benchmark)

Научное ядро представляет собой Python-пакет, реализующий всю вычислительную логику, связанную с проведением экспериментов, оценкой качества и статистической обработкой. Исходный код ядра опубликован в открытом репозитории [14] под лицензией MIT, что обеспечивает прозрачность и воспроизводимость результатов. Его структура представлена на рисунке 2.2.

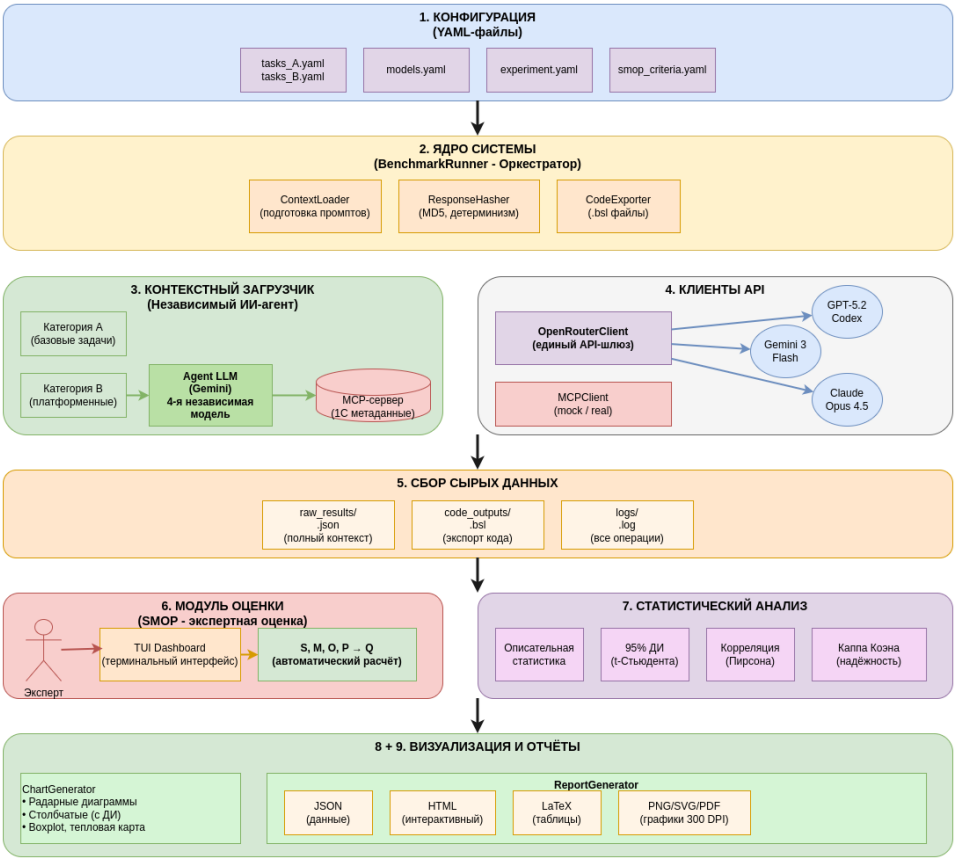


Рисунок 2.2 — Архитектура ядра платформы

2.3.2.1 Технологический стек ядра

Научное ядро реализовано на Python с использованием следующего набора библиотек:
— взаимодействие с API — requests, aiohttp;
— конфигурация — PyYAML, python-dotenv;
— схемы данных — pydantic, pydantic-settings;
— cli/tui интерфейсы — click, rich;

— статистика — numpy, scipy;
— визуализация — matplotlib;
— отчёты — jinja2;
— тестирование — pytest, pytest-asyncio.

Выбор Python обусловлен его широкими возможностями для научных расчётов, богатой экосистемой библиотек и простотой интеграции с различными API.

2.3.2.2 Структура и конфигурационные файлы

Организация репозитория ядра отражает функциональное разделение модулей и обеспечивает удобство настройки экспериментов, структура представлена на рисунке 2.3

```
AI-1C-Code-Gen-Benchmark/
├── configs/                          # Конфигурационные файлы
│   ├── models.yaml                   # Параметры моделей
│   ├── tasks_category_A.yaml         # Задачи категории A
│   ├── tasks_category_B.yaml         # Задачи категории B
│   └── smop_criteria.yaml            # Критерии оценки SMOP
├── src/                              # Исходный код
│   ├── clients/                      # Клиенты API
│   ├── core/                         # Основная логика
│   ├── evaluator/                    # Модуль оценки SMOP
│   └── utils/                        # Утилиты
├── code_outputs/                     # Сгенерированный код (.bsl)
├── raw_results/                      # Результаты экспериментов (JSON)
├── evaluations/                      # Экспертные оценки (JSON)
├── reports/                          # Отчёты и графики
├── logs/                             # Логи выполнения
├── main.py                           # Точка входа CLI
├── requirements.txt                  # Зависимости Python
└── README.md                         # Документация
```

Рисунок 2.3 — Структура ядра платформы

Конфигурационные файлы в формате YAML обеспечивают гибкость настройки без изменения программного кода:

— models.yaml содержит идентификаторы моделей в OpenRouter, параметры детерминированной генерации (температура, seed), лимиты контекста.

— tasks_category_*.yaml включают тексты задач, ожидаемые результаты и метаинформацию (категория, сложность).

— smop_criteria.yaml хранит детальные описания критериев для каждого уровня шкалы, что обеспечивает единообразие оценки при работе нескольких экспертов.

Такой подход позволяет проводить серии экспериментов с различными наборами моделей и задач, просто изменяя конфигурационные файлы.

2.3.2.3 Модуль генерации и взаимодействия с LLM

Модуль обеспечивает унифицированный доступ к языковым моделям через OpenRouter API и реализует агентный подход к сбору контекста метаданных при работе с MCP.

Базовое взаимодействие с OpenRouter.

Модуль включает клиент для работы с OpenRouter API, который обеспечивает:

— формирование запросов с учётом специфики каждой модели;

— управление параметрами детерминированной генерации (температура = 0, фиксированный seed);

— обработку сетевых ошибок и повторные попытки;

— сохранение метаинформации (токены, стоимость, время).

Агентный контекстный загрузчик (Agentic Context Loader)

Ключевой компонент для работы с MCP — агент, который динамически собирает контекст метаданных в процессе генерации. Его работа основана на следующем принципе: модель сама определяет, какая информация о структуре конфигурации ей необходима, и запрашивает её через MCP-инструменты.

Логика работы агента:

1. Получение доступных инструментов. При инициализации агент запрашивает у MCP-сервера список доступных tools через метод tools/list. Сервер возвращает описание инструментов в формате JSON-RPC.

2. Конвертация в формат OpenAI tools. Полученные описания преобразуются в формат, совместимый с API OpenAI (и поддерживаемый OpenRouter), что позволяет модели вызывать их как стандартные функции.

3. Итеративный сбор контекста. Агент передаёт модели исходную задачу вместе с описанием доступных инструментов. В процессе генерации модель может инициировать вызовы инструментов, которые проксируются через метод tools/call к MCP-серверу. Результаты каждого вызова возвращаются модели для дальнейшего анализа.

4. Завершение сбора. Процесс продолжается, пока модель не соберёт достаточный контекст для решения задачи либо не достигнет лимита итераций. Финальный ответ модели включает сгенерированный код с учётом полученных метаданных.

5. Кэширование результатов. Для повышения эффективности и снижения нагрузки на MCP-сервер агент поддерживает кэш запросов к инструментам. При повторном обращении к тому же объекту метаданных (например, структуре одного и того же справочника) результат возвращается из кэша без повторного вызова.

2.3.2.4 Модуль экспертной оценки SMOP

Модуль реализует методику SMOP, описанную в разделе 2.1, и обеспечивает полный цикл экспертной оценки:

1. Схемы данных на базе Pydantic: SMOPScores, RunEvaluation, TaskEvaluation, ExperimentEvaluation — обеспечивают типизированную работу с оценками и автоматическую валидацию.

2. SMOPEvaluator — менеджер сессий оценки, позволяющий:

— проставлять оценки по отдельности или пакетно;

— автоматически сохранять промежуточные результаты (в каталог evaluations/);

— возобновлять прерванную сессию с того же места;

— экспортировать результаты для статистической обработки.

Валидация гарантирует, что каждая оценка принадлежит множеству {0,2,4,6,8,10}, а все обязательные поля заполнены.

Интегральный показатель Q рассчитывается автоматически как среднее арифметическое четырёх критериев при любом их изменении.

Модуль не зависит от способа взаимодействия (TUI или веб) и может использоваться как в интерактивном, так и в пакетном режиме.

2.3.2.5 Модуль статистики и визуализации

Модуль обеспечивает количественный анализ результатов эксперимента и их графическое представление. Реализован в пакете src/evaluator/ и включает компоненты для статистической обработки, построения графиков и генерации отчётов.

Модуль statistics.py реализует следующий набор методов:

1. Описательные статистики:

— среднее арифметическое;
— медиана;
— стандартное отклонение (несмещённая оценка);
— минимум и максимум;
— квартили и межквартильный размах.

2. Доверительные интервалы. Для малых выборок используется t-распределение Стьюдента с автоматическим подбором критического значения в зависимости от размера выборки. Для выборок объёмом более 100 элементов применяется нормальное приближение ($z = 1.96$ для 95% доверительного интервала).

3. Корреляционный анализ. Для оценки связи между детерминизмом генерации (процент совпадающих ответов при повторных прогонах) и качеством кода (интегральный показатель Q) используется коэффициент корреляции Пирсона.

4. Межэкспертная надёжность. Для измерения согласованности между несколькими экспертами, оценивающими один и тот же код, используется Каппа Коэна (κ). Данный показатель учитывает не только наблюдаемое согласие, но и вероятность случайного совпадения оценок.

5. Агрегация данных. Модуль обеспечивает группировку результатов по различным измерениям:

— по моделям — для сравнения эффективности различных LLM;
— по задачам — для выявления типов заданий, вызывающих наибольшие сложности;
— по категориям задач (A и B) — для оценки влияния платформенного контекста;
— по режимам генерации (с MCP и без) — для количественной оценки эффекта от применения протокола.

6. Визуализация результатов. Модуль charts.py на базе библиотеки matplotlib генерирует научные графики, ориентированные на включение в публикации:

— Радарные диаграммы SMOP — отображают профиль каждой модели по четырём критериям, позволяя визуально сравнить сильные и слабые стороны (рисунок 2.3).
— Столбчатые диаграммы сравнения моделей — представляют интегральный показатель Q с доверительными интервалами и пороговыми линиями, соответствующими уровням качества.
— Тепловые карты — визуализируют значения Q для всех комбинаций «задача × модель», что позволяет быстро identify задачи, с которыми модели справляются хуже всего.
— Boxplot — показывают распределение оценок с указанием медианы, квартилей и выбросов.
— Диаграммы рассеяния — иллюстрируют корреляцию между детерминизмом генерации и качеством кода.

Все графики сохраняются в форматах PNG, SVG и PDF с разрешением 300 DPI, что соответствует требованиям большинства научных изданий.

7. Генерация отчётов. Модуль report.py создаёт итоговые артефакты в трёх форматах:

— JSON-отчёты — структурированные данные, включающие общую сводку по эксперименту, агрегированные статистики по моделям и задачам, детальные результаты по каждому прогону. Предназначены для дальнейшей автоматической обработки.
— HTML-отчёты — интерактивные веб-страницы с карточками ключевых метрик, таблицами сравнения и встроенными графиками. Обеспечивают удобный визуальный анализ результатов.
— LaTeX-таблицы — готовые таблицы для вставки в научные публикации, оформленные в соответствии с требованиями журналов.

Модуль статистики и визуализации получает входные данные от модуля экспертной оценки (через каталог evaluations/) и от модуля генерации (через raw_results/ для анализа детерминизма). Результаты его работы используются веб-платформой для отображения дашбордов и исследователем для подготовки публикаций.

2.3.2.6 Интерфейсы научного ядра

Ядро предоставляет два способа взаимодействия, обеспечивающие как пакетную обработку, так и интерактивную работу эксперта.

CLI (Command Line Interface) предназначен для автоматизации экспериментов и интеграции в скрипты. Реализован с использованием библиотеки click и поддерживает следующие команды:

python main.py run -c <категория> -m <модель> — запуск эксперимента для указанной категории задач (A или B) с выбранной моделью. Параметр --all-models позволяет последовательно запустить все доступные модели.

python main.py run -c B --mcp — запуск эксперимента с использованием MCP-сервера для задач категории B.

python main.py report <experiment_id> — построение отчёта по завершённому эксперименту с генерацией всех артефактов.

python main.py info --balance — проверка баланса OpenRouter API.

python main.py info --models — вывод списка доступных моделей с их характеристиками.

python main.py info --tasks A — просмотр описаний задач категории A.

CLI-интерфейс обеспечивает возможность пакетного запуска серий экспериментов, что особенно важно при необходимости многократных прогонов для оценки детерминизма (рисуноки 2.4, 2.5).

TUI (Terminal User Interface) для интерактивной экспертной оценки реализован с использованием библиотеки rich и предоставляет полноэкранный интерфейс в терминале (рисунок 2.6). Функциональные возможности TUI:

навигация по задачам и прогонам с визуализацией прогресса (цветовая индикация: зелёный — оценено, жёлтый — в процессе, серый — не начато);

отображение сгенерированного кода с подсветкой синтаксиса;

интерактивное проставление оценок по каждому критерию с использованием клавиатурных комбинаций;

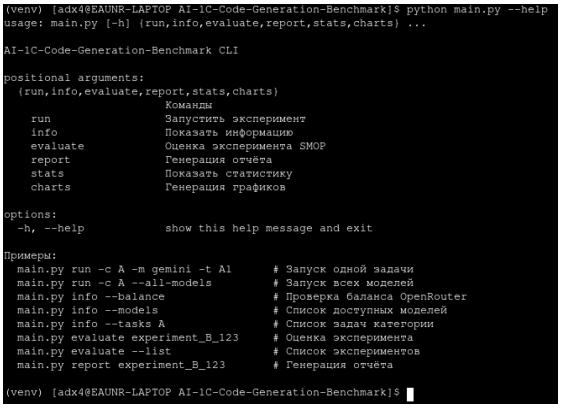
автоматический расчёт интегрального показателя Q при изменении любой из компонент;

добавление текстовых комментариев для обоснования оценки;

автосохранение состояния при выходе с возможностью возобновления сессии;

экспорт результатов в JSON для последующей статистической обработки.

TUI-интерфейс обеспечивает эффективную работу эксперта, минимизируя время на оценку каждого прогона и исключая ошибки ручного ввода.



```
(venv) [adx4@EAUNR-LAPTOP AI-1C-Code-Generation-Benchmark]$ python main.py --help
usage: main.py [-h] {run,info,evaluate,report,stats,charts} ...

AI-1C-Code-Generation-Benchmark CLI

positional arguments:
  {run,info,evaluate,report,stats,charts}
                        Команды
    run                 Запустить эксперимент
    info                Показать информацию
    evaluate            Оценка эксперимента SMOP
    report              Генерация отчёта
    stats               Показать статистику
    charts              Генерация графиков

options:
  -h, --help            show this help message and exit

Примеры:
  main.py run -c A -m gemini -t A1      # Запуск одной задачи
  main.py run -c A --all-models         # Запуск всех моделей
  main.py info --balance                # Проверка баланса OpenRouter
  main.py info --models                 # Список доступных моделей
  main.py info --tasks A                # Список задач категории
  main.py evaluate experiment_B_123     # Оценка эксперимента
  main.py evaluate --list               # Список экспериментов
  main.py report experiment_B_123       # Генерация отчёта

(venv) [adx4@EAUNR-LAPTOP AI-1C-Code-Generation-Benchmark]$
```

Рисунок 2.4 — Демонстрация точки входа CLI

```
[7/15] A3: Подсчёт рабочих дней между двумя датами
     Модель: Claude Opus 4.5
     [+] Прогон 1: no-seed, 543 токенов, 7.5c
     [+] Прогон 2: no-seed, 541 токенов, 6.7c
     [+] Прогон 3: no-seed, 543 токенов, 7.9c
     Итого: 1627 токенов, $0.0292, детерминизм: 67% (2 уник.)

[8/15] A3: Подсчёт рабочих дней между двумя датами
     Модель: GPT-5.2 Codex
     [+] Прогон 1: seed=178, 428 токенов, 4.9c
     [+] Прогон 2: seed=256, 445 токенов, 5.9c
     [+] Прогон 3: seed=349, 445 токенов, 5.2c
     Итого: 1318 токенов, $0.0132, детерминизм: 33% (3 уник.)

[9/15] A3: Подсчёт рабочих дней между двумя датами
     Модель: Gemini 3 Flash
     [+] Прогон 1: seed=178, 422 токенов, 3.5c
     [+] Прогон 2: seed=256, 422 токенов, 13.8c
     [+] Прогон 3: seed=349, 391 токенов, 2.8c
     Итого: 1235 токенов, $0.0027, детерминизм: 67% (2 уник.)

[10/15] A4: Парсинг JSON-строки
     Модель: Claude Opus 4.5
     [+] Прогон 1: no-seed, 1852 токенов, 20.4c
     [+] Прогон 2: no-seed, 1519 токенов, 16.0c
     [+] Прогон 3: no-seed, 4290 токенов, 37.3c
     Итого: 7661 токенов, $0.1799, детерминизм: 33% (3 уник.)

[11/15] A4: Парсинг JSON-строки
     Модель: GPT-5.2 Codex
     [+] Прогон 1: seed=178, 4244 токенов, 66.1c
     [+] Прогон 2: seed=256, 4244 токенов, 78.1c
```

Рисунок 2.5 — Демонстрация пакетной обработки CLI

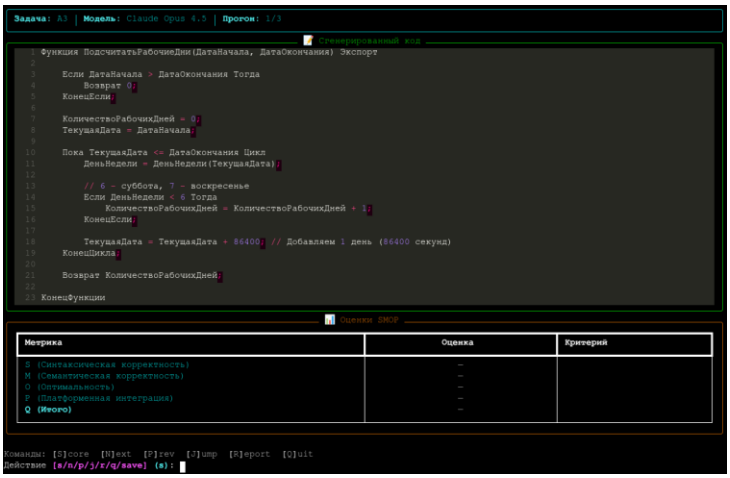


Рисунок 2.6 — Демонстрация интерактивной оценки кода TUI

2.3.3 Веб-платформа GenLab-1C

2.3.3.1 Технологический стек

Веб-платформа реализована на современном стеке технологий, обеспечивающем производительность, масштабируемость и удобство разработки:

— бекенд — FastAPI (Python)
— фронтенд — React (TypeScript);
— управление состоянием — Redux;
— база данных — Postgresql;
— ORM — SQLAlchemy;
— визуализация — recharts, d3.js;
— контейнеризация — Docker, Docker-Compose;
— тестирование — pytest, pytest-asyncio.

Выбор FastAPI обусловлен его высокой производительностью, автоматической генерацией OpenAPI-документации и простотой интеграции с Python-экосистемой научного ядра. React с TypeScript обеспечивает надёжность фронтенда и удобство поддержки кода.

2.3.3.2 Управление настройками эксперимента

Модуль конфигурирования платформы (рисунок 2.7), разделённый на логические блоки в соответствии с функциональными компонентами системы.

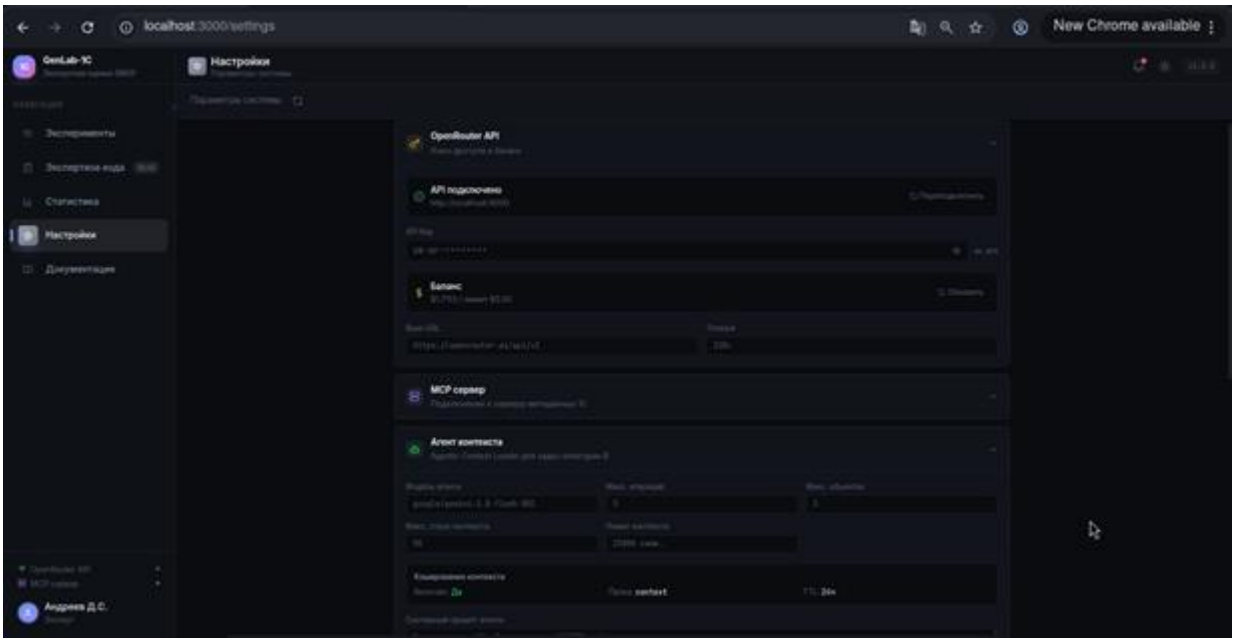


Рисунок 2.7 — Интерфейс управления настройками

Модуль конфигурирования платформы (рисунок 2.7) реализован в виде веб-интерфейса, разделённого на логические блоки в соответствии с функциональными компонентами системы:

OpenRouter API — настройки доступа к языковым моделям: поле для API-ключа с маскированием, отображение текущего баланса и лимита (прогресс-бар), кнопка обновления баланса, Base URL, таймаут соединения, статус подключения.

MCP сервер — настройки подключения к серверу метаданных 1С: URL сервера (по умолчанию http://localhost:8000), таймаут, статус подключения, кнопка переподключения.

Агент контекста — конфигурация Agentic Context Loader для задач категории B: выбор модели агента, максимальное количество итераций, максимальное количество объектов метаданных за вызов, лимиты контекста (строки и символы), настройки кэширования, редактируемые поля системного промпта и шаблона промпта агента.

Модели генерации — управление доступными моделями: список моделей с провайдерами и идентификаторами, переключатели активности, отображение количества активных моделей.

Шаблоны промптов — библиотека системных промптов для различных сценариев: список сохранённых шаблонов с назначением, индикатор активного шаблона, возможность добавления, редактирования и удаления, выбор шаблона по умолчанию для каждой категории задач.

Хеширование — настройки для обеспечения воспроизводимости: выбор алгоритма хеширования (MD5, SHA-256), возможность включения дополнительных полей в хеш.

2.3.3.3 Управление тестовыми заданиями

Интерфейс управления тестовыми заданиями включает следующие компоненты:

Панель инструментов:

1. Выпадающий список для выбора существующего эксперимента
2. Кнопка обновления списка экспериментов и статуса запуска
3. Кнопка создания нового эксперимента

Основной контент:

1. Баннер статуса запущенного эксперимента (отображается прогресс, текущая задача, модель, стоимость, токены, время выполнения)

2. Детальная информация выбранного эксперимента: сводка (категория, количество моделей и задач, токены, стоимость), таблица параметров генерации по моделям, раскрываемый список задач с промптами и результатами

3. Пустое состояние с предложением создать эксперимент

Модальное окно создания эксперимента:

1. Выбор режима (категория А — алгоритмические задачи, категория В — платформенные задачи, кастомный режим)
2. Настройка параметров генерации (temperature, max tokens, количество прогонов)
3. Выбор моделей и задач
4. Добавление/редактирование пользовательских задач (для кастомного режима)
5. Кнопка запуска эксперимента

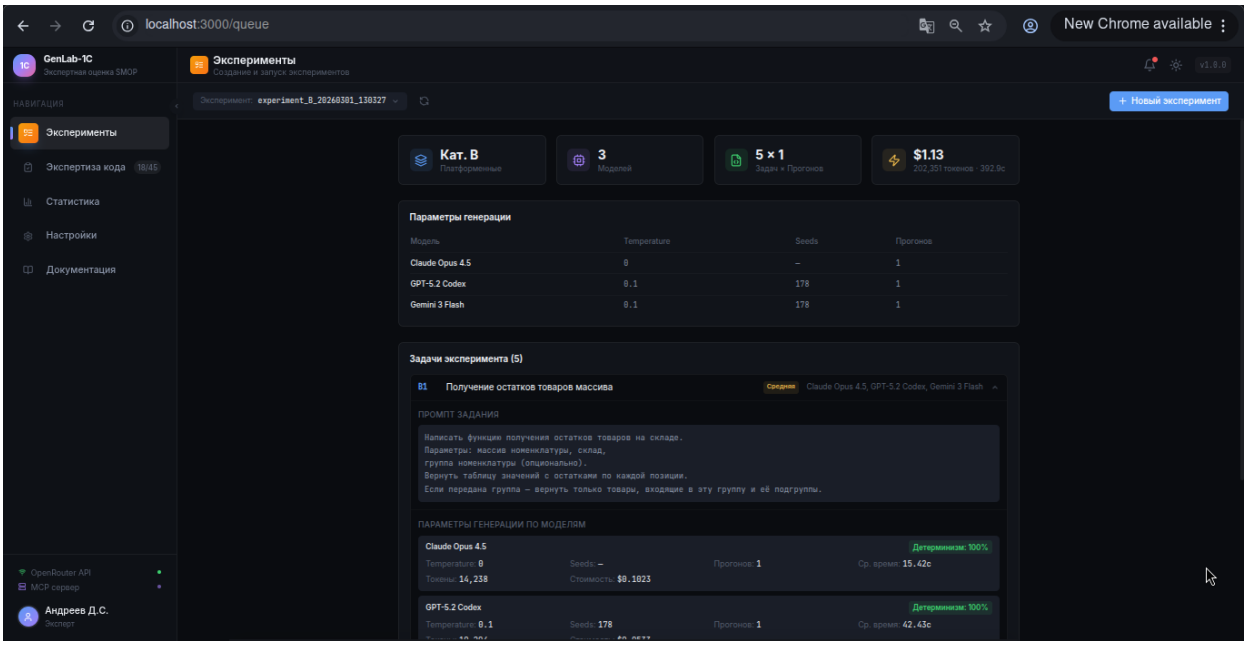


Рисунок 2.8 — Интерфейс просмотра заданий

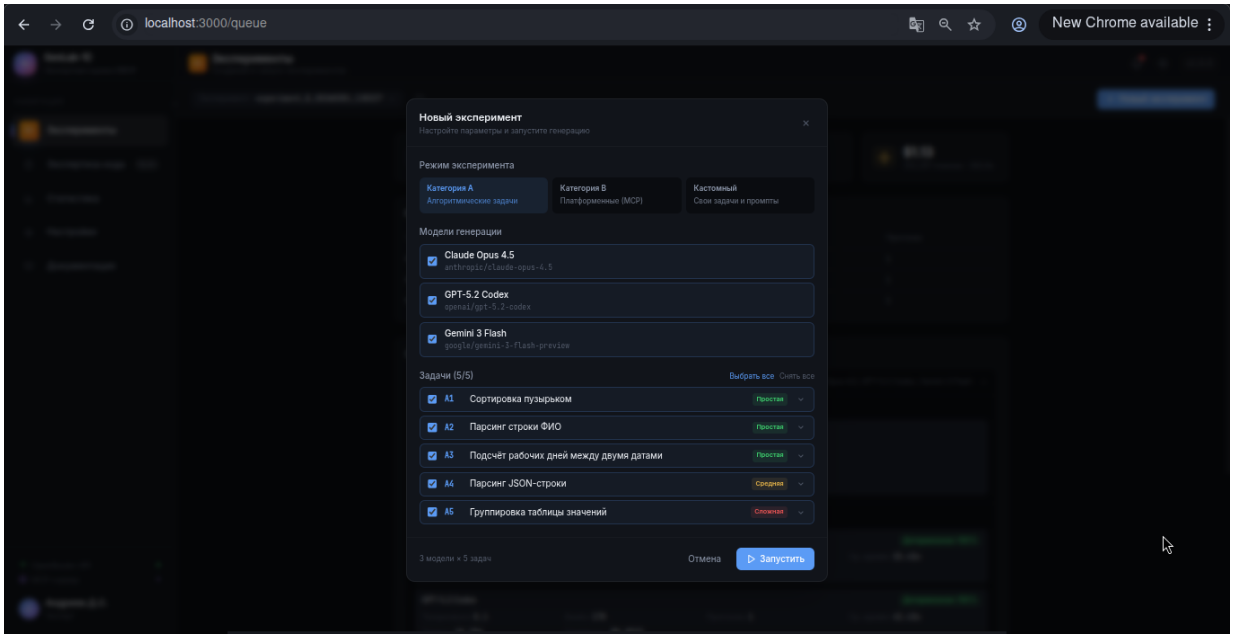


Рисунок 2.9 — Интерфейс настроек заданий

2.3.3.4 Коллаборативная экспертная оценка

Интерфейс коллективной экспертной оценки реализован на странице Экспертиза кода и обеспечивает многопользовательский анализ результатов экспериментов с возможностью проставления оценок сгенерированному коду.

Боковая панель навигации содержит список доступных экспериментов, позволяя эксперту переключаться между ними.

Основная панель отображает детальную информацию по выбранной задаче:

Заголовок задачи содержит идентификатор, название, категорию и использованную модель, а также раскрываемый блок с полным промптом задачи.

Просмотр кода реализует токенизированный вывод с подсветкой синтаксиса языка 1С, нумерацией строк и возможностью копирования. Для каждого сгенерированного решения отображаются метаданные: идентификатор задачи, модель, параметры генерации (температура, прогон), количество токенов, стоимость, время выполнения и показатель детерминизма (процент совпадений между запусками).

Панель запусков представляет список всех прогонов задачи с возможностью переключения между ними. Для каждого запуска отображается номер и оценка качества с цветовой индикацией (зеленый/желтый/красный). Справа выводится индикатор детерминизма с процентом совпадений и количеством уникальных ответов.

Интерфейс обрабатывает состояния загрузки, ошибок и отсутствия экспериментов с соответствующими визуальными индикаторами и кнопками повторных попыток.

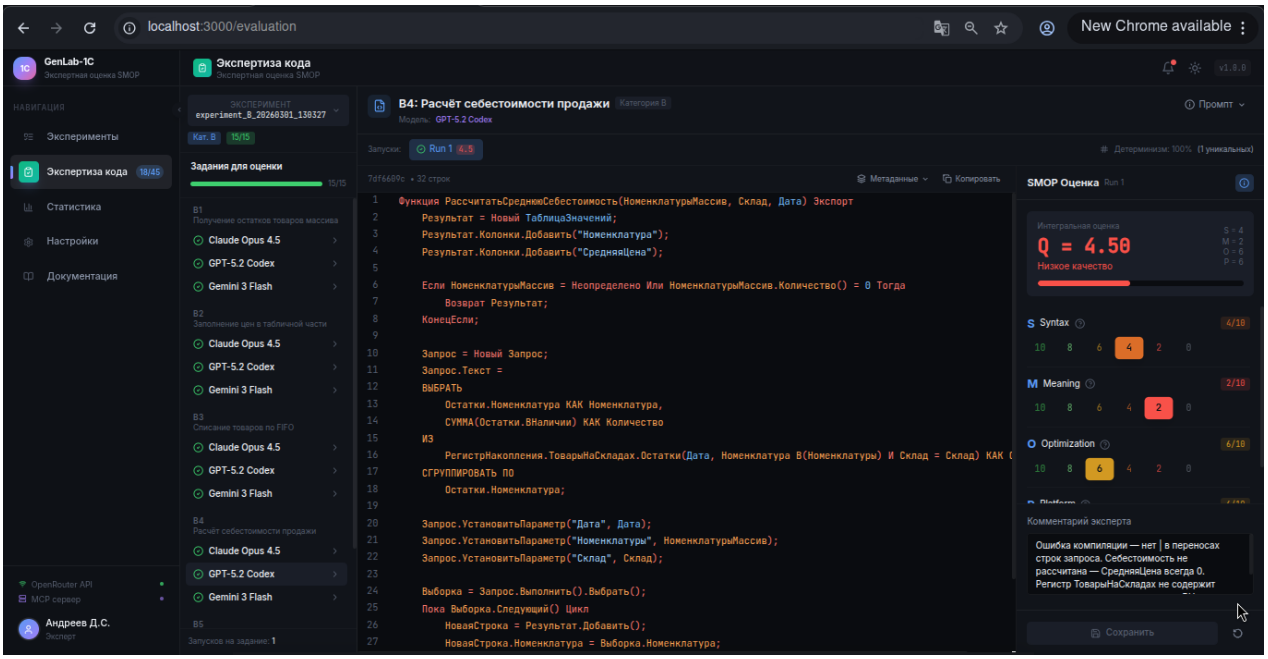


Рисунок 2.10 — Интерфейс экспертной оценки кода

2.3.3.5. Визуализация результатов и дашборды

Страница Статистика реализует комплексную визуализацию результатов экспериментов с набором интерактивных дашбордов.

Панель инструментов содержит выпадающий список для выбора эксперимента (с отображением идентификатора, категории, количества задач и моделей) и кнопки экспорта отчетов в форматах JSON, HTML и LaTeX.

Карточки статистики отображают ключевые метрики:

- среднее значение интегральной метрики качества (Q);
- стандартное отклонение (σ);

- количество оцененных запусков из общего числа;
- 95% доверительный интервал.

Визуализация данных включает:

1. Гистограмма SMOP — сравнение моделей по четырем метрикам (S — синтаксис, M — семантика, O — оптимальность, P — платформенная интеграция) с цветовой кодировкой.
2. Радарная диаграмма — многомерное сравнение моделей по метрикам SMOP.
3. Интегральный Q по моделям — горизонтальные полосы со цветовой индикацией (зеленый — $Q \geq 8$, желтый — $5 \leq Q < 8$, красный — $Q < 5$).
4. Детерминизм моделей — полосы, отображающие процент совпадений между запусками (зеленый — ≥80%, желтый — 50–80%, красный — <50%).
5. Тепловая карта (Heatmap) — матрица "задачи × модели" с цветовым кодированием среднего Q.
6. Корреляция детерминизм ↔ качество — числовое значение коэффициента корреляции (r).

Интерфейс обрабатывает состояния загрузки, ошибок и отсутствия данных, поддерживает экспорт графиков в формате SVG и динамическое обновление при выборе эксперимента.

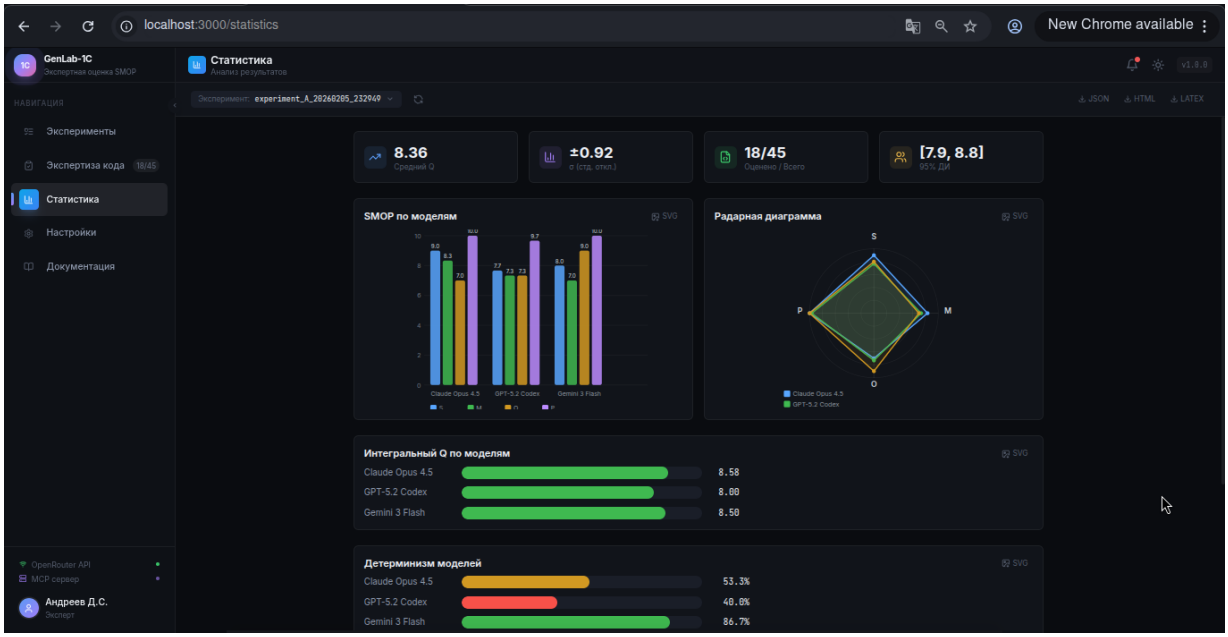


Рисунок 2.11 — Интерфейс метрик эксперимента

2.3.3.6. Веб документация методики SMOP

Страница Документация представляет собой интерактивное руководство по методике оценки качества кода, обеспечивающее прозрачность и воспроизводимость результатов.

Верхняя панель содержит иконку книги и заголовок "Документация платформы", идентифицируя текущий раздел.

Основной контент включает:

Описание платформы GenLab-1C: краткая информация о назначении (бенчмарки генерации кода на 1С:Предприятие 8.3 с использованием LLM), поддержке MCP и разделении задач на категории A (без контекста) и B (с контекстом).

Методика SMOP: описание четырёх метрик оценки:

- S (Syntax) — синтаксическая корректность;
- M (Meaning) — семантическая корректность;
- O (Optimization) — оптимальность и производительность;
- P (Platform) — платформенная интеграция (работа с объектами метаданных).

Дискретная шкала оценивания: шкала от 0 до 10 баллов с шагом 2, исключающая возможность "средней" оценки и вынуждающая эксперта принимать однозначное решение. Цветовая кодировка: зелёный (8–10) — высокое качество, жёлтый (4–6) — среднее, красный (0–2) — низкое.

Таблицы критериев: для каждой метрики приведены конкретные критерии оценки с привязкой к баллам и цветовой подсветкой.

Интегральный показатель качества Q: формула расчета Q = (S + M + O + P) / 4 и интерпретация значений (Q ≥ 8 — высокое качество, 5 ≤ Q < 8 — приемлемое, Q < 5 — низкое).

Страница выполняет обучающую функцию, обеспечивая единообразное понимание критериев оценки всеми участниками эксперимента.

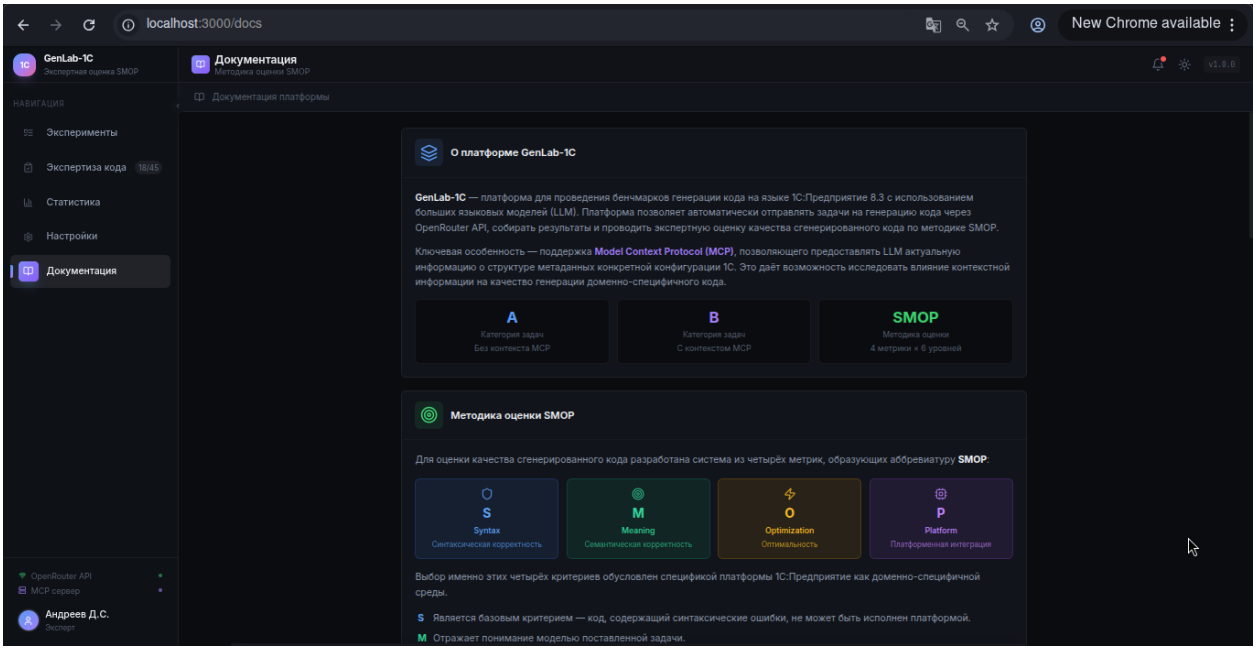


Рисунок 2.12 — Интерфейс документации

2.3.4 Интеграция ядра и веб-платформы

Интеграция между открытым научным ядром и веб-платформой реализована через REST API, где ядро выступает в роли вычислительного сервера, а веб-платформа — клиента.

Схема взаимодействия:

1. Веб-платформа принимает конфигурацию эксперимента от пользователя через веб-интерфейс.
2. Веб-платформа отправляет POST-запрос к API ядра с конфигурацией в формате JSON.
3. Ядро выполняет генерацию кода, сохраняет результаты в файловой системе и возвращает идентификатор эксперимента.
4. Веб-платформа периодически опрашивает статус выполнения через API (эндпоинт /status/{exp_id}).
5. По завершении ядро возвращает ссылки на результаты (JSON-файлы и каталоги с кодом).
6. Веб-платформа читает результаты и отображает пользователю.

Принципы интеграции:

1. Независимость ядра. Ядро не имеет зависимостей от веб-платформы и может использоваться полностью автономно через CLI/TUI. Это обеспечивает:

— воспроизводимость — исследователь может запустить ядро локально и получить те же результаты;

— прозрачность — весь код ядра открыт для проверки;

— надёжность — сбой веб-платформы не влияет на возможность проведения экспериментов.

2. Асинхронность. Все длительные операции (генерация кода, статистическая обработка) выполняются асинхронно с сохранением состояния в БД.

3. Безопасность. Доступ к API ограничен JWT-токенами, все запросы логируются, чувствительные данные (ключи API) хранятся в зашифрованном виде.

4. Расширяемость. API спроектирован с возможностью добавления новых эндпоинтов без нарушения обратной совместимости.

Такая архитектура обеспечивает гибкость использования: исследователь может работать как через удобный веб-интерфейс, так и напрямую с ядром, будучи уверенным в идентичности результатов.

2.4. Тестовая инфраструктура

Для автоматизации проведения эксперимента и обеспечения воспроизводимости результатов использовался разработанный в рамках данного исследования программный комплекс GenLab-1C (версия 1.0), включающий открытое научное ядро для генерации кода и веб-платформу для коллективной экспертной оценки.

Таблица 2.1 — Программное обеспечения для проведения исследования

Компонент	Версия и описание
Платформа для исследования	1С:Предприятие 8.3 (8.3.26.1656) Версия для разработчиков.
Конфигурация	Управление торговлей, редакция 11 (11.5.17.234)
MCP-сервер	vladimir-kharin/1c_mcp, версия 1.3
Операционная система	Windows 10 Pro, ArchLinux
Среда разработки	1С:Enterprise Development Tools (EDT)
Программный комплекс GenLab-1C	Open Source научное ядро AI-1C-Code-Gen-Benchmark (версия 1.0) + веб-платформа GenLab-1C

Для исследования использовался рекомендуемый вариант подключения через Python-прокси, обеспечивающий поддержку STDIO-транспорта. Расширение MCP_Сервер.cfe установлено в тестовую информационную базу с конфигурацией УТ11.

Валидация корректности настроек. Перед началом эксперимента проведена проверка работоспособности инфраструктуры:

— тестовый запрос к MCP-серверу на получение списка доступных инструментов (tools/list);

— пробная генерация кода для задачи B1 с использованием MCP;

— проверка сохранения результатов в файловой системе.

Все проверки завершились успешно, инфраструктура признана готовой к проведению эксперимента.

2.5. Дизайн эксперимента

2.5.1 Тестовый набор задач

На основе анализа типовых задач 1С-разработки [6, 7] сформирован набор из 10 тестовых заданий, разделённых на две категории по степени зависимости от контекста конфигурации.

В таблице 2.2 представлены базовые задачи категории A, не требующие знания структуры конкретной конфигурации. Могут быть решены на основе общих знаний о синтаксисе языка BSL.

Таблица 2.2 — Набор тестовых задач категории A

№	Задача	Описание
A1	Сортировка массива	Реализация алгоритма сортировки массива чисел методом пузырька на языке BSL
A2	Работа со строками	Парсинг строки формата «Фамилия Имя Отчество» и формирование структуры с полями ФИО и инициалами
A3	Работа с датами	Вычисление количества рабочих дней между двумя датами с исключением выходных (суббота, воскресенье)
A4	Чтение JSON	Парсинг JSON-строки с вложенной структурой и преобразование в объекты платформы (Структура, Массив)
A5	Работа с таблицей значений	Группировка строк таблицы значений по указанной колонке с подсчётом суммы по числовому полю

В таблице 2.3 представлены прикладные задачи категории B, требующие понимания структуры конфигурации УТ11: имён регистров, реквизитов документов, событийной модели платформы.

Таблица 2.3 — Набор тестовых задач категории B

№	Задача	Описание
B1	Получение остатков товаров массива	Функция получения остатков на складе с фильтрацией по группе номенклатуры
B2	Заполнение цен в табличной части	Процедура заполнения колонки «Цена» в табличной части «Товары» документа из регистра сведений «ЦеныНоменклатуры»
B3	Списание товаров по FIFO	Процедура списания товаров со склада методом FIFO с контролем достаточности остатков и формированием таблицы движений по партиям
B4	Расчёт себестоимости продажи	Функция расчёта себестоимости проданных товаров по методу средневзвешенной стоимости на основании данных регистра партий
B5	Пересчёт валютных сумм	Процедура пересчёта сумм документа при смене валюты или курса с обновлением табличной части

2.5.2 Исследуемые модели и параметры генерации

В эксперименте исследовались три языковые модели, доступные через OpenRouter API. Выбор моделей обусловлен их высокой производительностью в задачах генерации кода и доступностью через единый интерфейс. В таблице 2.4 представлены характеристики моделей.

Таблица 2.4 — Характеристики исследуемых моделей

Модель	ID OpenRouter	Контекст	Параметры генерации
Claude Opus 4.5	anthropic/claude-opus-4.5	200K	temperature=0, max_tokens=4096

GPT-5.2 Codex	openai/gpt-5.2-codex	400K	tempreature=0.1, seed=[178, 256, 349], max_tokens=4096
Gemini 3 Flash	google/gemini-3-flash-preview	1M	tempreature=0.1, seed=[178, 256, 349], max_tokens=4096

Параметры генерации унифицированы для всех моделей с учётом их особенностей:

1. Температура установлена в 0 для Claude Opus 4.5 и в 0.1 для GPT-5.2 Codex и Gemini 3 Flash. Такие значения обеспечивают детерминированность генерации, минимизируя случайные вариации ответов, что критически важно для воспроизводимости эксперимента.

2. Зерно (seed) фиксировано для моделей, поддерживающих этот параметр. Для GPT-5.2 Codex и Gemini 3 Flash используется набор из трёх значений [178, 256, 349], соответствующих трём прогонам каждой задачи. Это позволяет:

— оценить детерминированность модели (степень совпадения ответов при разных seed);

— выявить случаи, когда модель даёт различные, но равноценные решения;

— обеспечить возможность точного воспроизведения каждого прогона.

3. Максимальная длина ответа установлена в 4096 токенов для всех моделей, что достаточно для генерации решений на BSL (средний объём кода для задач категории B не превышает 300–500 строк).

Системные промпты. Для каждой категории задач используется отдельный системный промпт, учитывающий специфику решаемых заданий.

Для категории A (алгоритмические задачи) промпт фокусируется на синтаксической корректности и готовности кода к выполнению:

«Разработка для платформы 1С:Предприятие 8.3.
ТРЕБОВАНИЯ К КОДУ:
1. Пишите на встроенном языке 1С (русский синтаксис)
2. Возвращайте ТОЛЬКО код без пояснений
3. Код должен быть готов к выполнению»

Для категории B (платформенно-зависимые задачи) промпт дополнен указаниями по работе с MCP и конфигурацией УТ11:

«Разработка для платформы 1С:Предприятие 8.3.
Конфигурация: Управление торговлей 11 (УТ11).
КРИТИЧЕСКИ ВАЖНО:
1. Используйте ТОЛЬКО объекты метаданных через MCP
2. НЕ выдумывайте несуществующие реквизиты, справочники или регистры
3. Пишите на встроенном языке 1С (русский синтаксис)
4. Возвращайте ТОЛЬКО код без пояснений
5. Код должен быть готов к выполнению»

Промпт для агентного контекстного загрузчика. При работе в режиме с MCP дополнительно используется специализированный промпт, оптимизирующий процесс сбора метаданных:

«Ты эксперт по 1С. Твоя задача — БЫСТРО найти нужные метаданные для написания кода.
ПРАВИЛА:
1. Запроси МАКСИМУМ 2-3 объекта метаданных (регистр, справочник или документ)
2. Как только понял структуру нужных объектов — СРАЗУ вызови finish_research
3. НЕ запрашивай лишние объекты "на всякий случай"
4. Для запроса остатков достаточно: РегистрНакопления + 1 связанный справочник
Вызови finish_research как можно раньше!
Задача:
{task_prompt}
Найди 1-3 нужных объекта метаданных и СРАЗУ вызови finish_research.»

Такой подход минимизирует количество вызовов MCP-инструментов и ускоряет процесс генерации, что особенно важно при пакетном запуске экспериментов.

Оценка детерминизма. Для каждой комбинации «задача — модель» выполняется три прогона с идентичными параметрами (фиксированная температура, различные seed для поддерживающих моделей). Для каждого прогона вычисляется SHA-256 хеш ответа. Процент совпадающих хешей в трёх прогонах используется как мера детерминированности модели. Данный показатель в дальнейшем применяется в корреляционном анализе для выявления связи между стабильностью генерации и качеством кода.

2.5.3 Сводные параметры эксперимента

Для обеспечения полноты и воспроизводимости исследования разработан комплексный дизайн эксперимента, ключевые параметры которого сведены в таблицу 2.5.

Таблица 2.5 — Сводные параметры дизайна эксперимента

Параметр	Значение
Задачи	
Количество задач категории A	5
Количество задач категории B	5
Всего задач	10
Модели	
Исследуемые модели	Claude Opus 4.5, GPT-5.2 Codex, Gemini 3 Flash
Прогонов на задачу	3
Объем эксперимента	
Генераций категории A	5 задач × 3 модели × 3 прогона = 45
Генераций категории B	5 задач × 3 модели × 3 прогона = 45
Общее число генераций	90
Экспертиза	
Количество экспертов	3

Представленный дизайн обеспечивает решение следующих исследовательских задач:

Сравнение моделей — за счёт выполнения одних и тех же задач всеми тремя моделями в идентичных условиях.

Анализ детерминизма — за счёт трёхкратного повторения каждой комбинации с фиксированными параметрами.

Минимизация субъективности — благодаря использованию трёх независимых экспертов и формализованной шкалы оценивания.

Обеспечение воспроизводимости — через хеширование ответов, сохранение полной конфигурации и детерминированные параметры генерации.

Такой подход позволяет получить статистически значимые результаты и сформировать обоснованные выводы об эффективности исследуемых моделей качество генерации кода для платформы 1С:Предприятие.

ГЛАВА 3. РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТАЛЬНОЙ АПРОБАЦИИ

3.1. Категория A (задачи без контекста метаданных)

3.1.1 Общие показатели

Для задач категории A, не требующих знания структуры конкретной конфигурации, все три модели продемонстрировали высокие результаты. В таблице 3.1 представлены средние значения по каждой метрике SMOP для трёх моделей.

Таблица 3.1 — Средние значения метрик SMOP для задач категории A

Модель	S	M	O	P	Q	σ_Q
Claude Opus 4.5	9.2	9.6	8.4	9.2	9.1	0.52
GPT-5.2 Codex	8.0	8.0	7.5	7.0	7.63	0.85
Gemini 3 Flash	9.2	9.6	9.6	9.2	9.4	0.45

Как видно из таблицы, наилучший интегральный показатель Q продемонстрировала модель Gemini 3 Flash (9.40), незначительно опередив Claude Opus 4.5 (9.10). GPT-5.2 Codex показал существенно более низкие результаты (7.63), что может быть связано с особенностями модели при работе с русскоязычным синтаксисом BSL.

По синтаксической корректности (S) и семантической корректности (M) лидируют Claude Opus 4.5 и Gemini 3 Flash (9.2–9.6). Наибольшее отставание GPT-5.2 Codex наблюдается по метрике платформенной интеграции (P) — 7.0 против 9.2 у конкурентов.

На рисунке 3.1 представлена радарная диаграмма, визуализирующая профили моделей по четырём критериям SMOP для задач категории A.

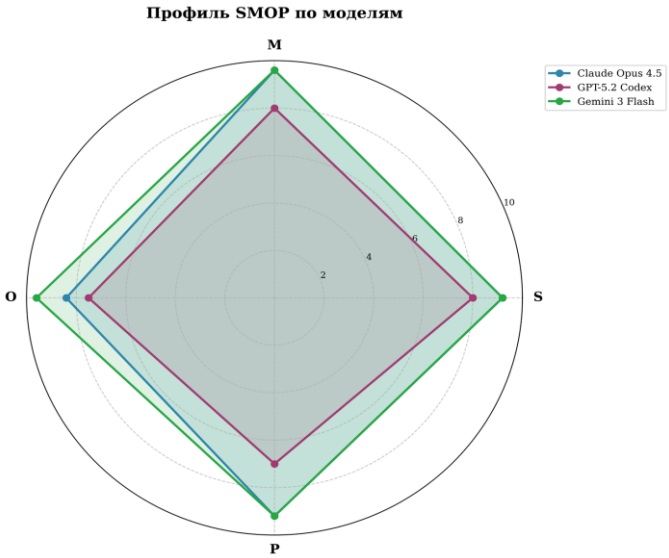


Рисунок 3.1 — Радарная диаграмма категории A

На радарной диаграмме наглядно видно, что профили моделей Claude Opus 4.5 и Gemini 3 Flash практически совпадают по трём метрикам (S, M, P), однако Gemini вырывается вперёд по оптимальности (O). Профиль GPT-5.2 Codex имеет меньшую площадь, что соответствует более низким значениям по всем метрикам, с особенно заметным провалом по метрике P.

3.1.2 Детальный анализ метрик

Для более глубокого понимания сильных и слабых сторон каждой модели обратимся к тепловой карте, представленной на рисунке 3.2. Данная визуализация показывает распределение интегрального показателя качества Q по отдельным задачам (A1–A5) для каждой модели. Цветовая кодировка соответствует принятой шкале: зелёный цвет (Q ≥ 8) — высокое качество, жёлтый (5 ≤ Q < 8) — среднее качество, красный (Q < 5) — низкое качество. Прочерк («—») означает отсутствие результата (модель не сгенерировала решение).

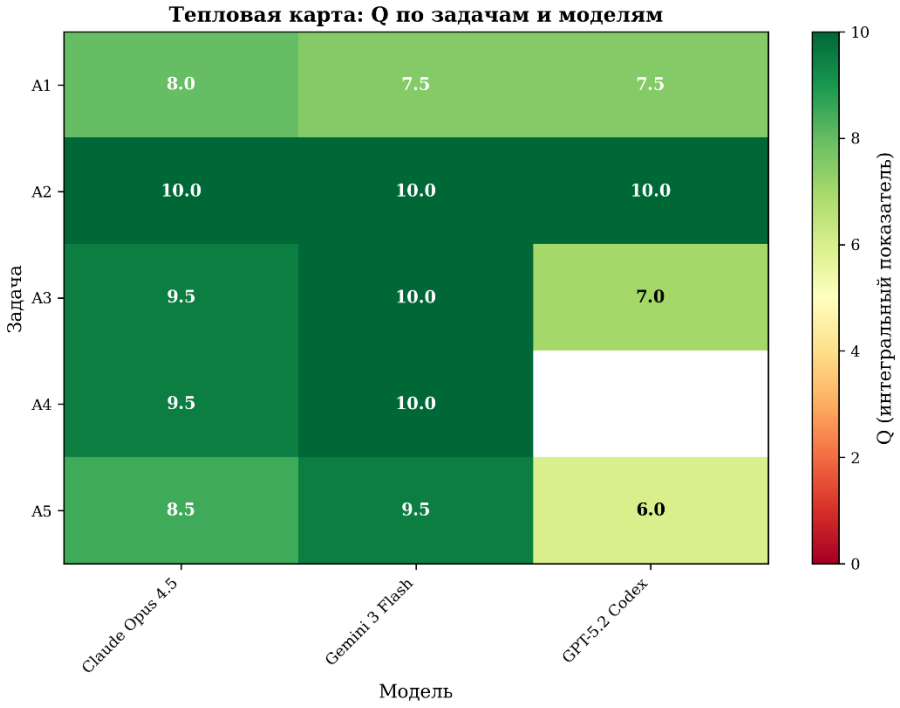


Рисунок 3.2 — Тепловая карта для категории A

Задача A1 (работа с массивом) оказалась сложной для всех трёх моделей. Все они допустили одну и ту же синтаксическую ошибку — использование кириллической переменной «и» в качестве счётчика цикла, которая конфликтует с ключевым словом языка «И». Это привело к снижению оценок по синтаксису (S = 6) и платформенной интеграции (P = 6) у всех моделей. Gemini 3 Flash дополнительно допустила семантическую ошибку, использовав неглубокое копирование (Знач вместо клонирования), что модифицировало исходный массив. В результате все три модели попали в «жёлтую зону» (Q = 7.5–8.0).

Задача A2 (парсинг ФИО) не вызвала затруднений ни у одной из моделей. Все три показали идеальный результат (Q = 10.0), корректно обработав краевые случаи: полное ФИО, отсутствие отчества, только фамилия, пустая строка, множественные пробелы. Gemini 3 Flash отметилась наиболее компактной реализацией.

Задача A3 (рабочие дни) выявила различия в подходах. Gemini 3 Flash применила оптимальный формульный подход (Q = 10.0), в то время как Claude Opus 4.5 использовала линейный перебор, что привело к снижению оценки по оптимальности (O = 8) и итоговому Q = 9.5. GPT-5.2 Codex допустила критические семантические ошибки: прибавление секунды вместо дня и некорректную обработку обратного порядка дат, получив Q = 7.0.

Задача A4 (парсинг JSON) продемонстрировала провал GPT-5.2 Codex — модель не сгенерировала решение (прочерк). Claude Opus 4.5 предложила две реализации (включая

избыточный ручной парсер), что снизило оптимальность (O = 8), но в целом результат высокий (Q = 9.5). Gemini 3 Flash оптимально использовала встроенный ПрочитатьJSON (Q = 10.0).

Задача A5 (группировка таблицы значений) вновь показала отставание GPT-5.2 Codex. Модель допустила ошибку выполнения (несуществующий метод СодержитКлюч) и использовала неоптимальную ручную группировку (Q = 6.0). Claude Opus 4.5 и Gemini 3 Flash справились успешно (Q = 8.5 и 9.5 соответственно), хотя обе использовали избыточные операции (ручная группировка у Claude, цикл удаления колонок у Gemini).

Обобщая анализ по задачам:

1. Gemini 3 Flash показывает наиболее стабильные результаты: только одна задача (A1) попала в жёлтую зону, остальные четыре — зелёные с максимальными или близкими к максимуму оценками.

2. Claude Opus 4.5 также стабилен: все пять задач в зелёной зоне, однако по трём задачам (A1, A3, A4) имеет незначительные потери качества из-за неоптимальных решений.

3. GPT-5.2 Codex демонстрирует крайне нестабильные результаты: одна задача не решена вовсе (A4), две задачи в жёлтой зоне (A1, A5), одна на границе (A3) и лишь одна идеальная (A2). Это подтверждает высокий разброс оценок (σ = 0.85), отмеченный ранее.

3.1.3 Ранжирование моделей по качеству

Для наглядного сравнения моделей по отдельным метрикам SMOP на рисунке 3.3 представлена столбчатая диаграмма, детализирующая вклад каждой метрики в интегральную оценку качества.

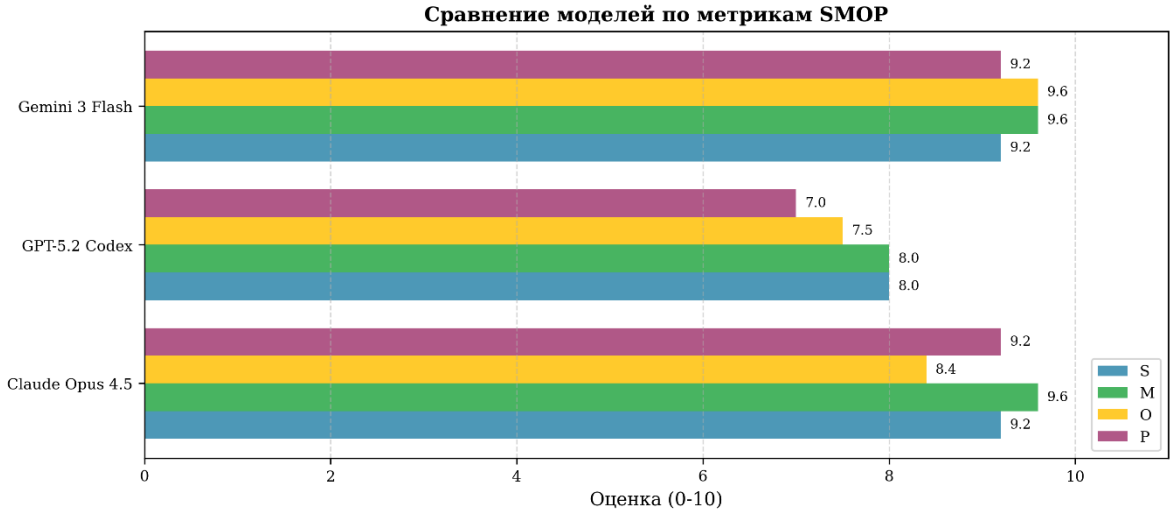


Рисунок 3.3 — Сравнение моделей по метрикам категории A

Анализ диаграммы позволяет выделить сильные и слабые стороны каждой модели:

1. Gemini 3 Flash демонстрирует наиболее сбалансированный и высокий профиль. Модель лидирует по метрике оптимальности (O = 9.6), существенно опережая Claude Opus 4.5 (O = 8.4), и делит первенство с Claude по синтаксису (S = 9.2) и семантике (M = 9.6). Платформенная интеграция (P = 9.2) также на высшем уровне. Такой профиль подтверждает, что Gemini одинаково хорошо справляется как с алгоритмической корректностью, так и с производительностью кода.

2. Claude Opus 4.5 показывает близкие к лидеру значения по трём метрикам (S, M, P), однако заметно уступает по оптимальности (O = 8.4). Это системная особенность модели: в

задачах A3 (рабочие дни) и A4 (парсинг JSON) Claude предлагала рабочие, но неоптимальные решения (линейный перебор вместо формулы, ручной парсер вместо встроенного механизма). При этом семантическая корректность (M = 9.6) — сильнейшая сторона модели.

3. GPT-5.2 Codex существенно отстаёт по всем метрикам. Наибольшее падение наблюдается по платформенной интеграции (P = 7.0) — модель часто использует конструкции, не свойственные языку 1С (например, несуществующий метод СодержитКлюч в задаче A5). Синтаксис (S = 8.0) и семантика (M = 8.0) также заметно ниже конкурентов. Единственная метрика, где разрыв не столь катастрофичен — оптимальность (O = 7.5), однако это скорее следствие того, что многие решения GPT просто не доходят до этапа оптимизации из-за синтаксических или семантических ошибок.

Ранжирование моделей по интегральному показателю Q (рассчитанному ранее в таблице 3.1) выглядит следующим образом:

- Gemini 3 Flash (Q = 9.40) — абсолютный лидер, стабильно высокие результаты по всем метрикам.
- Claude Opus 4.5 (Q = 9.10) — уверенный второй, с небольшим отставанием по оптимальности.
- GPT-5.2 Codex (Q = 7.63) — аутсайдер, требующий доработки для применения в 1С-разработке.

Таким образом, для задач категории A (не требующих знания конфигурации) наиболее предпочтительной моделью является Gemini 3 Flash, обеспечивающая наилучший баланс корректности и производительности. Claude Opus 4.5 может быть рекомендован как надёжная альтернатива там, где критична семантическая точность. Использование GPT-5.2 Codex в текущем виде сопряжено с высокими рисками некачественной генерации

3.2. Категория Б (платформенно-ориентированные задачи)

3.2.1 Общие показатели

В таблице 3.2 представлены средние значения по каждой метрике SMOP для трёх моделей.

Таблица 3.2 — Средние значения метрик SMOP для задач категории Б

Модель	S	M	O	P	Q	σ_Q
Claude Opus 4.5	8.0	8.4	9.2	8.0	8.4	0.89
GPT-5.2 Codex	4.0	4.8	5.2	5.6	4.9	1.71
Gemini 3 Flash	6.4	8.0	7.6	6.4	7.10	1.2

Примечание: σ_Q рассчитано на основе данных по отдельным задачам (B1–B5)

Как видно из таблицы, в задачах категории B, требующих понимания структуры метаданных конфигурации и использования MCP, лидерство сохраняет Claude Opus 4.5 с интегральным показателем Q = 8.40. Модель демонстрирует уверенное преимущество по всем метрикам, особенно по оптимальности (O = 9.2), что указывает на эффективное использование контекста метаданных для построения производительных решений.

Gemini 3 Flash занимает вторую позицию (Q = 7.10), существенно отставая от лидера по синтаксису (S = 6.4) и платформенной интеграции (P = 6.4). При этом семантическая корректность (M = 8.0) остаётся на высоком уровне — модель понимает, что нужно сделать, но допускает ошибки в реализации.

GPT-5.2 Codex показывает критически низкие результаты (Q = 4.90). Модель не справляется с платформенными задачами даже при наличии контекста метаданных. Наибольший разброс оценок (σ = 1.71) подтверждает крайнюю нестабильность: от частично рабочих решений (B2, B5) до полностью неработоспособных (B3, где генерация оборвалась).

На рисунке 3.4 представлена радарная диаграмма, визуализирующая профили моделей по четырём критериям SMOP для задач категории B.

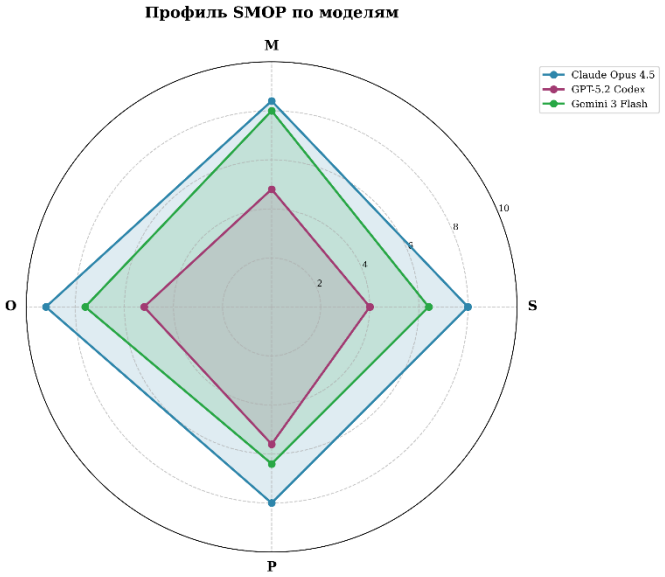


Рисунок 3.4 — Радарная диаграмма категории B

Радарная диаграмма наглядно демонстрирует профили моделей. Claude Opus 4.5 образует почти правильный четырёхугольник с небольшим смещением в сторону оптимальности — сбалансированный профиль высокого уровня. Gemini 3 Flash имеет характерную «звезду» с вытянутым лучом по семантике (M) и провалами по синтаксису и платформе. GPT-5.2 Codex представляет собой маленький четырёхугольник, смещённый в сторону платформы — модель иногда угадывает нужные объекты метаданных (P = 5.6), но не способна построить синтаксически корректный код.

3.2.2 Детальный анализ метрик

Для более глубокого понимания сильных и слабых сторон каждой модели обратимся к тепловой карте, представленной на рисунке 3.5.

Анализ тепловой карты позволяет выявить следующие закономерности:

Задача B1 (остатки товаров) — Claude Opus 4.5 показывает почти идеальный результат (Q = 9.0), незначительно снизив оценку из-за хрупкого приёма СтрЗаменить для модификации запроса. Gemini 3 Flash также успешно справляется (Q = 8.0), хотя допустила синтаксическую избыточность (фигурные скобки {} из СКД). GPT-5.2 Codex демонстрирует низкий результат (Q = 5.5) с множеством ошибок: неверное имя ресурса, потеря аналитик, неоптимальные условия.

Задача B2 (упаковка номенклатуры) — все модели столкнулись с типичной для 1С проблемой: использование не типизированных колонок таблицы значений в запросе. Это привело к синтаксическим ошибкам у всех трёх моделей (S = 6). Claude Opus 4.5 (Q = 8.0) выигрывает за счёт оптимальной структуры запроса и корректной логики. Gemini 3 Flash (Q = 7.0) и GPT-5.2 Codex (Q = 6.5) отстают из-за ошибок в работе с составными ключами и захардкоженных значений.

Задача B3 (FIFO-списание) — наиболее сложная задача категории. Claude Opus 4.5 (Q = 9.0) предлагает полностью рабочее решение с пакетным запросом, хотя и с неточностью в логике FIFO (сортировка по дате производства вместо партий поступления). Gemini 3 Flash (Q = 5.0) создала нерабочую реализацию с ошибками выполнения. GPT-5.2 Codex (Q = 2.0) сгенерировала обрезанный код без завершающих конструкций — критический провал.

Задача B4 (себестоимость) — Gemini 3 Flash (Q = 8.0) показывает лучший результат, корректно реализовав средневзвешенную, но допустив опечатку в ключевом слове запроса (ГРУППИРОВАТЬ вместо СГРУППИРОВАТЬ). Claude Opus 4.5 (Q = 7.0) выбрала неверный регистр для расчёта себестоимости. GPT-5.2 Codex (Q = 4.5) не решила задачу — себестоимость всегда равна нулю.

Задача B5 (кросс-курс валют) — Claude Opus 4.5 (Q = 9.0) демонстрирует близкое к идеальному решение с корректной формулой и округлением. Gemini 3 Flash (Q = 7.5) справляется, но с потенциально опасным использованием SetParam и обращением к несуществующим реквизитам. GPT-5.2 Codex (Q = 6.0) предложила логически верное, но синтаксически нерабочее решение (вызов методов регистров вне запроса).

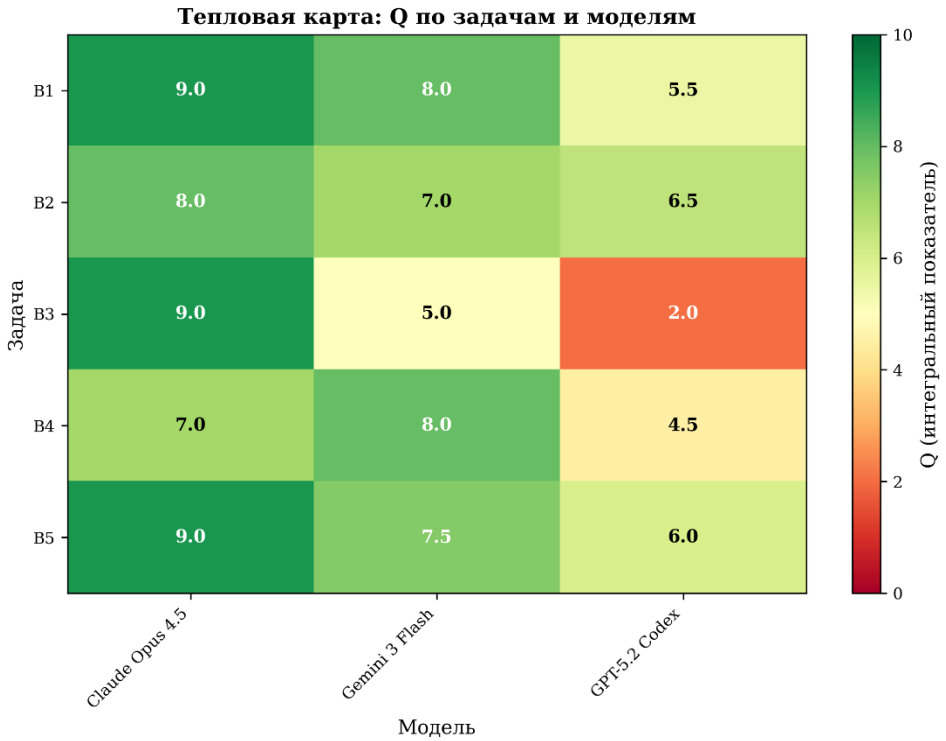


Рисунок 3.5 — Радарная диаграмма категории B

3.2.3 Ранжирование моделей по качеству

Для наглядного сравнения моделей по отдельным метрикам обратимся к столбчатой диаграмме.

Claude Opus 4.5 уверенно лидирует по всем метрикам. Особенно выделяется оптимальность (O = 9.2) — модель эффективно использует контекст MCP для построения производительных запросов и алгоритмов. Семантическая корректность (M = 8.4) также на высоте: Claude понимает бизнес-логику задач. Отставание по синтаксису (S = 8.0) и платформенной интеграции (P = 8.0) связано с отдельными ошибками (нетипизированные колонки, хрупкие приёмы), но в целом уровень высокий.

Gemini 3 Flash показывает неоднородный профиль. Сильная сторона — семантика (M = 8.0), модель правильно понимает, что требуется сделать. Синтаксис (S = 6.4) и платформенная интеграция (P = 6.4) — слабые места: модель часто путает синтаксис запроса (фигурные скобки, ГРУППИРОВАТЬ), допускает ошибки в именах объектов метаданных и не всегда корректно использует контекст.

GPT-5.2 Codex демонстрирует крайне низкие результаты по всем метрикам. Наиболее близка к приемлемому уровню платформенная интеграция (P = 5.6) — модель иногда угадывает нужные объекты, но не может правильно их использовать. Синтаксис (S = 4.0) и семантика (M = 4.8) на критически низком уровне.

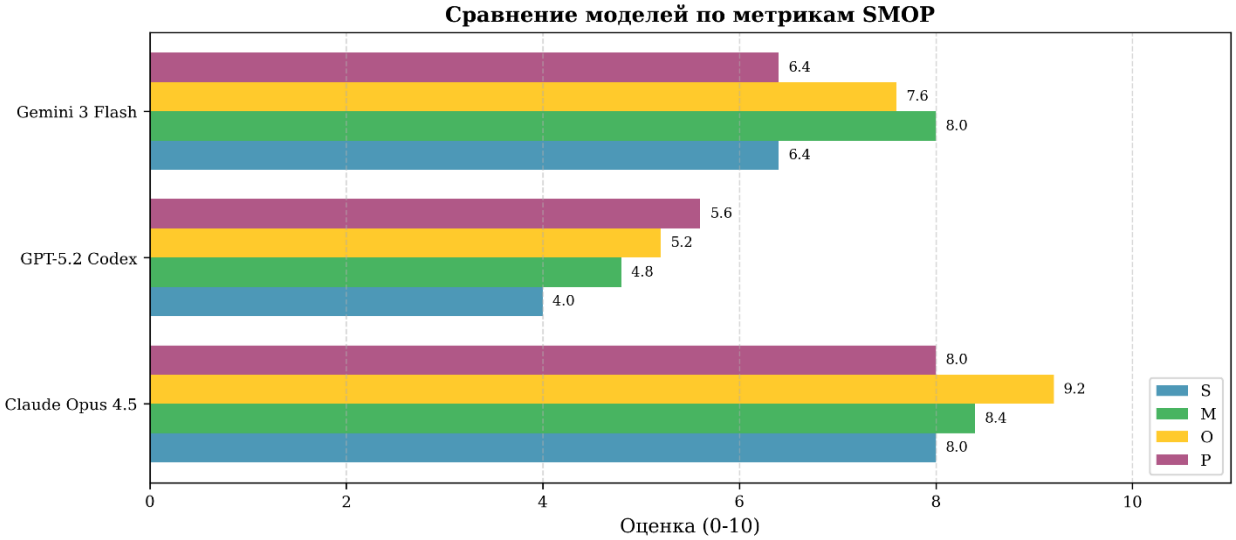


Рисунок 3.6 — Сравнение моделей по метрикам категории B

3.3. Сравнительный анализ категорий A и B

3.3.1 Влияние MCP на качество генерации

Для оценки влияния контекста метаданных, предоставляемого через MCP, сопоставим результаты моделей в категориях A (без контекста) и B (с контекстом). В таблице 3.3 представлены интегральные показатели качества Q для обеих категорий.

Таблица 3.3 — Сравнение интегрального качества Q в категориях A и B

Модель	Категория A	Категория Б	Δ (B − A)
Claude Opus 4.5	9.10	8.4	−0.70
GPT-5.2 Codex	7.63	4.9	−2.73
Gemini 3 Flash	9.4	7.1	−2.30

Полученные результаты демонстрируют парадоксальную на первый взгляд картину: все три модели показали снижение качества в категории B по сравнению с категорией A. Наибольшее падение продемонстрировал GPT-5.2 Codex (−2.73), наименьшее — Claude Opus 4.5 (−0.70).

Этот результат требует содержательной интерпретации. Снижение качества не означает, что MCP «ухудшает» генерацию. Оно отражает принципиально разную сложность задач:

Категория A включала алгоритмические задачи, не привязанные к конкретной

конфигурации (парсинг ФИО, работа с массивами, подсчёт рабочих дней). Это задачи, на которых языковые модели традиционно сильны, так как они близки к массовому обучающему материалу.

Категория B требовала не только алгоритмического мышления, но и глубокого понимания структуры метаданных конфигурации Управление торговлей 11 (регистры, документы, табличные части, особенности платформы). Это задачи, для которых у моделей значительно меньше обучающих примеров.

Таким образом, падение Q при переходе от A к B — это мера сложности платформенных задач относительно алгоритмических, а не мера эффективности MCP. Для корректной оценки вклада MCP необходимо сравнивать категорию B не с A, а с гипотетической категорией B' (те же платформенные задачи, но без MCP). К сожалению, в рамках данного эксперимента такие данные отсутствуют.

3.3.2 Анализ причин падения качества

Рассмотрим, по каким именно метрикам произошло наибольшее снижение.

Таблица 3.4 — Изменение метрик SMOP при переходе от A к B (Δ = B − A)

Модель	ΔS	ΔM	ΔO	ΔP
Claude Opus 4.5	−1.2	−1.2	+0.8	−1.2
GPT-5.2 Codex	−4.0	−3.2	−2.3	−1.4
Gemini 3 Flash	−2.8	−1.6	−2.0	−2.8

Анализ изменений по метрикам позволяет сделать следующие наблюдения:

Claude Opus 4.5 — единственная модель, показавшая прирост по оптимальности (ΔO = +0.8) в категории B. Это прямое свидетельство эффективного использования контекста MCP: получив доступ к структуре метаданных, Claude смогла строить более оптимальные запросы и алгоритмы. Падение по остальным метрикам (S, M, P) связано со сложностью платформенных задач, но оно минимально среди всех моделей.

Gemini 3 Flash продемонстрировала равномерное падение по всем метрикам (Δ = −1.6…−2.8). Наибольшее снижение — по синтаксису (ΔS = −2.8) и платформенной интеграции (ΔP = −2.8). Это указывает на то, что модель, хорошо справляясь с алгоритмической сутью задач (семантика упала меньше всего — ΔM = −1.6), испытывает трудности с корректным синтаксическим оформлением платформенных конструкций и точным использованием объектов метаданных.

GPT-5.2 Codex показала катастрофическое падение по синтаксису (ΔS = −4.0) и семантике (ΔM = −3.2). Модель фактически не справляется с платформенными задачами — в категории B её результаты находятся в «красной зоне» (Q = 4.90). Интересно, что падение по платформенной интеграции (ΔP = −1.4) наименьшее, но это связано с тем, что и в категории A этот показатель был низким (P = 7.0).

3.3.3 Выводы по сравнительному анализу

Платформенные задачи существенно сложнее алгоритмических для всех исследованных моделей. Среднее падение Q по трём моделям составило 1.91 балла (по 10-балльной шкале).

Claude Opus 4.5 наиболее эффективно использует контекст MCP — единственная модель, улучшившая оптимальность решений в категории B. Падение качества минимально, что позволяет рекомендовать эту модель для платформенной разработки 1С с использованием MCP.

Gemini 3 Flash, будучи лидером в алгоритмических задачах, значительно теряет позиции на платформенных задачах из-за синтаксических ошибок и неточностей в работе с метаданными.

GPT-5.2 Codex непригодна для решения платформенных задач 1С в текущем состоянии — качество генерации находится на неприемлемо низком уровне.

Полученные результаты подтверждают необходимость разработки специализированных методик оценки (таких как SMOP) и использования контекстных протоколов (MCP) для повышения качества генерации в доменно-специфических областях.

3.4. Ограничения исследования

При интерпретации полученных результатов необходимо учитывать следующие ограничения проведённого эксперимента:

1. Размер выборки задач. Исследование использовало по 5 задач в каждой категории, всего 10 задач. Хотя задачи были тщательно отобраны для покрытия ключевых аспектов 1С-разработки и приближены к реальным сценариям (работа с остатками, FIFO-списание, расчёт себестоимости, кросс-курсы валют), ограниченный размер выборки не позволяет делать статистически обобщённые выводы о качестве моделей на всём пространстве задач платформы 1С:Предприятие. При этом общий объём генераций составил 90 (45 в каждой категории), что обеспечивает достаточную базу для первичных выводов.

2. Одна конфигурация метаданных. Все задачи категории B выполнялись в контексте одной конфигурации — «Управление торговлей 11» (УТ 11). Различные конфигурации (Бухгалтерия, Управление холдингом, ERP, Зарплата и управление персоналом) имеют существенно различающуюся структуру метаданных, и результаты могут не переноситься напрямую на другие конфигурации.

3. Отсутствие прямого сравнения «с MCP / без MCP». В силу дизайна эксперимента задачи категории B выполнялись только с использованием MCP. Это не позволяет количественно измерить вклад именно протокола MCP в улучшение качества — мы можем лишь констатировать абсолютные значения качества для платформенных задач, но не прирост относительно генерации без контекста.

4. Ограничения агентного режима загрузки контекста. При работе с MCP использовался специализированный агентный контекстный загрузчик с промптом, ограничивающим количество запрашиваемых объектов метаданных (максимум 2–3 объекта на задачу: регистр, справочник или документ). Это решение было принято для минимизации времени выполнения и снижения токенов, однако оно потенциально ограничивает полноту контекста: в реальных сценариях сложные задачи могут требовать обращения к 5–10 объектам метаданных. Таким образом, результаты отражают эффективность MCP при ограниченном, минимально достаточном контексте, а не при полном доступе ко всем метаданным конфигурации.

5. Отсутствие автоматизированной тестовой среды на стороне 1С. В рамках эксперимента не использовались специализированные фреймворки автоматизированного тестирования, такие как Vanessa Automation или 1С:Тестировщик. Проверка корректности сгенерированного кода выполнялась экспертом вручную, включая анализ синтаксиса, семантики, оптимальности и платформенной интеграции. Это ограничивает возможность автоматического прогона тестов при увеличении объёма задач и вносит элемент субъективности в оценки по метрикам O и M.

6. Фиксированные параметры генерации. Использование temperature = 0 (максимальный детерминизм) является стандартной практикой для бенчмарков, однако в реальных сценариях разработки часто используются ненулевые значения температуры для повышения креативности решений. Результаты могут отличаться при других параметрах генерации. Все запросы обрабатывались в отдельных потоках без сохранения контекста беседы, что соответствует условиям однократных вызовов API.

7. Ограниченный набор моделей. Исследование включало три модели, доступные на момент проведения эксперимента (Claude Opus 4.5, GPT-5.2 Codex, Gemini 3 Flash). Появление новых версий моделей (Claude Opus 4.6, GPT-5.3, Gemini 3.5 Flash и др.) или специализированных fine-tune под 1С может изменить наблюдаемые соотношения.

ЗАКЛЮЧЕНИЕ

В ходе выполнения научно-исследовательской работы была разработана и апробирована методика многокритериальной оценки качества генерации кода для платформы 1С:Предприятие, основанная на метриках SMOP (Syntax, Meaning, Optimization, Platform). Для автоматизации процесса оценки создан программный комплекс GenLab-1C, интегрирующий доступ к языковым моделям через OpenRouter, поддержку протокола MCP для получения контекста метаданных и веб-интерфейс для коллективной экспертной оценки.

Экспериментальная апробация методики включала 10 задач (5 алгоритмических — категория A, 5 платформенных с использованием MCP — категория B), выполненных тремя моделями (Claude Opus 4.5, GPT-5.2 Codex, Gemini 3 Flash). Общий объём генераций составил 90, каждая из которых была оценена экспертом по четырём метрикам SMOP по дискретной шкале {0, 2, 4, 6, 8, 10} с последующим расчётом интегрального показателя Q.

Проведённое исследование позволяет сформулировать следующие основные выводы:

1. Для алгоритмических задач (категория A) наилучший результат показала модель Gemini 3 Flash ($Q = 9.40$), незначительно опередив Claude Opus 4.5 ($Q = 9.10$). GPT-5.2 Codex существенно отстаёт ($Q = 7.63$), демонстрируя нестабильность результатов ($\sigma = 0.85$) и низкую платформенную интеграцию даже в задачах, не требующих знания конфигурации.

2. Для платформенных задач с MCP (категория B) лидерство сохраняет Claude Opus 4.5 ($Q = 8.40$), единственная модель, попавшая в «зелёную зону» высокого качества. Gemini 3 Flash ($Q = 7.10$) показывает приемлемые результаты, но теряет позиции из-за синтаксических ошибок и неточностей в работе с метаданными. GPT-5.2 Codex ($Q = 4.90$) непригодна для решения платформенных задач 1С.

3. Сравнительный анализ категорий выявил значительное снижение качества при переходе от алгоритмических задач к платформенным (среднее падение $Q = 1.91$ балла), что отражает принципиально более высокую сложность задач, требующих понимания структуры метаданных. При этом Claude Opus 4.5 продемонстрировала прирост оптимальности ($\Delta O = +0.8$) в категории B, что свидетельствует об эффективном использовании контекста MCP.

Разработанная методика SMOP показала свою пригодность для многокритериального анализа качества генерации. Дискретная шкала с шагом 2 балла обеспечивает однозначность экспертных решений, а разделение на четыре независимые метрики позволяет локализовать источники ошибок (синтаксические, семантические, оптимизационные, платформенные).

Программный комплекс GenLab-1C подтвердил работоспособность предложенного подхода, обеспечивая интеграцию с языковыми моделями, загрузку контекста метаданных через MCP, проведение экспериментов и визуализацию результатов (радарные диаграммы, тепловые карты, сравнение метрик).

Полученные результаты и выявленные в ходе работы ограничения позволяют наметить перспективные направления дальнейших исследований:

1. Расширение набора задач до 20–30 в каждой категории с сохранением их приближенности к реальным сценариям разработки для повышения статистической значимости выводов.

2. Включение нескольких конфигураций (Бухгалтерия, ERP, Управление холдингом) для проверки обобщаемости результатов на различные структуры метаданных.

3. Проведение эксперимента в дизайне «с MCP / без MCP» для количественной оценки вклада протокола в повышение качества генерации платформенного кода.

4. Оптимизация агентного загрузчика контекста — исследование влияния объёма предоставляемых метаданных (2–3 объекта против 5–10) на точность и полноту решений.

5. Разработка автоматизированной тестовой среды на базе Vanessa Automation для интеграции в контур оценки, что позволит автоматически проверять функциональную корректность сгенерированного кода.

6. Привлечение большего пула экспертов (5–7 человек) с расчётом межэкспертной согласованности для минимизации субъективности оценок.

7. Анализ влияния различных параметров генерации (temperature, top_p, presence_penalty) на качество и креативность решений.

8. Включение большего числа моделей, включая российские (GigaChat, YandexGPT) и специализированные fine-tune под 1С, для сравнительного анализа с лидерами рынка.

9. Интеграция RAG-подхода (Retrieval-Augmented Generation) для обогащения контекста.

10. Интеграция с инструментами разработчика — создание плагинов для EDT и VS Code, позволяющих разработчикам получать рекомендации по улучшению кода на основе анализа SMOP-метрик прямо в среде разработки.

Реализация указанных направлений позволит создать полноценную систему объективной оценки качества генерации кода для платформы 1С:Предприятие и выработать практические рекомендации по эффективному использованию языковых моделей в задачах автоматизации разработки.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. SWE-Bench Verified Leaderboard // LLM Stats. — 2025. — URL: https://llm-stats.com/benchmarks/swe-bench-verified (дата обращения: 22.12.2025).

2. SWE-bench // Vals AI. — 2025. — URL: https://www.vals.ai/benchmarks/swebench (дата обращения: 22.12.2025).

3. Российский рынок ERP-систем: итоги 2024 года и перспективы // TAdviser. — 2025. — URL: https://www.tadviser.ru/index.php/Статья:Центры_компетенций_1С_2024._Обзор_TAdviser (дата обращения: 24.02.2026).

4. Kocetkov D., Nguyen T., Li Y., Ainslie J., Roziere B., Lample G. The Stack: 3 TB of permissively licensed source code // arXiv:2211.15533. — 2022. — URL: https://arxiv.org/abs/2211.15533 (дата обращения: 29.12.2025).

5. Introducing the Model Context Protocol // Anthropic. — 2024. — URL: https://www.anthropic.com/news/model-context-protocol (дата обращения: 07.01.2026).

6. Вайб кодинг в 1С. Лучшие нейросети для генерации 1С кода // Habr. — 2025. — URL: https://habr.com/ru/articles/908170/ (дата обращения: 07.01.2026).

7. Как с помощью MCP-серверов решать проблемы генерации ИИ-кода в экосистеме 1С // BPMStart. — 2025. — URL: https://bpmstart.com/... (дата обращения: 09.01.2026).

8. Specification — Model Context Protocol. — 2025. — URL: https://modelcontextprotocol.io/specification/2025-06-18 (дата обращения: 07.01.2026).

9. Chen Y., Li A., Smith J., Brown E., Wang R. Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions // arXiv:2503.23278. — 2025. — URL: https://arxiv.org/abs/2503.23278 (дата обращения: 07.01.2026).

10. vladimir-kharin/1c_mcp: Разработка MCP-серверов в 1С // GitHub. — 2025. — URL: https://github.com/vladimir-kharin/1c_mcp (дата обращения: 09.01.2026).

11. Салихов Д. 1С Code Bench — бенчмарк для оценки способности LLM-моделей писать код на 1С // FastCode. — 2025. — URL: https://1cbench.github.io/ (дата обращения: 20.02.2026).

12. Бобров В. BenchLabs — бенчмарк LLM-моделей для 1С:Предприятие // BenchLabs. — 2025. — URL: https://benchlabs.ru/ (дата обращения: 20.02.2026).

13. Харин В. Тестирование свободно распространяемых LLM в задачах разработки и архитектуры 1С // GitHub. — 2025. — URL: https://github.com/vladimir-kharin/llm_1c_benckmark (дата обращения: 20.02.2026).

14. 4dand / AI-1C-Code-Gen-Benchmark // GitHub. — 2026. — URL: https://github.com/4dand/AI-1C-Code-Gen-Benchmark (дата обращения: 01.03.2026).